Reti di calcolatori

Alessandro Carella

Indice

1. Reti di calcolatori e Internet 4
   1.1. Storia di Internet 4
   1.2. Gli «ingranaggi» di Internet 4
      1.2.1. I protocolli 5
   1.3. I componenti della rete 5
      1.3.1. Le reti di acceso 5
   1.4. Il nucleo della rete 5
      1.4.1. Commutazione di pacchetto 5
      1.4.2. Commutazione di circuito 6
      1.4.3. Una rete di reti 8
   1.5. Tipi di reti 8
   1.6. Ritardi, perdite e throughput nelle reti a commutazione di pacchetto ... 9
      1.6.1. Ritardo nelle reti a commutazione di pacchetto 9
2. Architetture a livelli 10
   2.1. Stratificazione dei protocolli 10
   2.2. Incapsulamento 13
3. Livello di applicazione 13
   3.1. Web e HTTP 13
      3.1.1. Architettura del Browser e del Server 14
      3.1.2. Differenza tra URI, URN e URL 14
      3.1.3. Panoramica di HTTP 14
      3.1.4. HTTP/1 15
      3.1.5. Connessioni persistenti e non persistenti 16
      3.1.6. Formato dei messaggi HTTP 17
      3.1.7. Interazione utente-server: i cookie 20
      3.1.8. Web caching 21
      3.1.9. HTTP/2 22
      3.1.10. HTTP/3 22
      3.1.11. Streaming video e reti per la distribuzione di contenuti 23
   3.2. DNS: il servizio directory di Internet 27
      3.2.1. Servizi forniti da DNS 27
      3.2.2. Panoramica del funzionamento di DNS 29
      3.2.3. Record e messaggi DNS 32
      3.2.4. Registrazione di un dominio e Zone Transfer 34
4. Livello di Trasporto 35
   4.1. Introduzione e servizi a lviello di trasporto 35

4.1.1. Relazione tra i livello di trasporto e di rete 35
4.1.2. Panoramica del livello di trasporto di Internet 36
4.2. Multiplexing e demultiplexing 36
4.2.1. Multiplexing e demultiplexing non orientati alla connessione ... 38
4.2.2. Multiplexing e demultiplexing orientati alla connessione 39
4.3. Trasporto non orientato alla connessione: UDP 40
4.3.1. Struttura dei segmenti UDP 41
4.3.2. Checksum UDP 42
4.4. Trasporto orientato alla connessione: TCP 43
4.4.1. Connessione TCP 44
4.4.2. Struttura dei segmenti TCP 45
4.4.3. Timeout e stima del tempo di andata e ritorno (RTT) 48
4.4.4. Trasferimento dati affidabile 49
4.4.5. Controllo di flusso 55
4.4.6. Gestione della connessione TCP 56
4.4.7. Controllo di congestione TCP 59
4.4.8. Notifica esplicita di congestione dalla rete 64
4.4.9. Fairness 64
4.5. QUIC: Quick UDP Internet Connection 65

Reti di calcolatori e Internet

1.1. Storia di Internet

1.2. Gli «ingranaggi» di Internet

Internet è una rete di calcolatori che interconnette miliardi di dispositivi di calcolo in tutto il mondo. Spesso ci si connette a Internet tramite smartphone e tablet, anche se, oggi, molti più dispositivi come TV, console di gicoo, automobili, elettrodomestici, ecc. vengono via via connesse. Tutti questi dispositivi sono detti **host** o **sistemi periferici** (*end systems*). I sistemi periferici sono connessi tra loro tramite una **rete di collegamenti** (*communication link*) e **commutatori di pacchetti** (*packet switch*). I collegamenti possono essere di molti tipi, costituiti da varie tipologie di mezzi fisici, come cavi coassiali. fili di rame, fibre ottiche, ecc. Collegamenti diversi trasmettono dati a velocità differenenti, e tale **velocità di trasmissione** viene misurata in but/secondo (bps). Quando un host vuole inviare dati a un altro sistema periferico, suddivide i dati in sottoparti e aggiunge un'intestazione a ciascune di esse: l'insieme delle informazioni risultati, viene chiamato **pacchetto**. Un commutatore di pacchetto prende un pacchetto che arriva da uno dei collegamenti in ingresso e lo ritrasmette su uno di quelli in uscita. I princiapli commutatori di pacchetto sono i **router** e i **commutatori a livello di collegamento** (*link-layer switch*). Dal sistema di invio a quello di ricezione, la sequenza di collegamenti e di commuttatori di pacchetto attraversata dal singolo pacchetto è nota come **percorso** (*route* o *path*) attraverso la rete.
I sistemi perferici accedono a Internet tramite i cosiddetti **Internet Service Provider (ISP)**. Un provider è un insieme di commutatori di pacchetto e di collegamenti. GLi ISP forniscono agli host svariati tipi di acceso alla rete, tra cui quello residenziale a larga banda come la DSL, quello in rete locale ad alta velocità e quello wireless.
Sistemi perferici, commutatori di pacchetto e altre parti di Internet fanno uso di protocolli che controllano l'invio e la ricezione di informazioni all'interno della rete. Due dei principali protocolli Internet sono il **Transmission Control Protocol (TCP)** e l'**Internet Protocol (IP)**. Quyest'ultimo specifica il formato dei pacchetti scambiati tra router e sistemi periferici. I principali protocolli Internet sono noti con il nome collettivo di **TCP/IP**.

1.2.1. I protocolli

Diamo una definizione specifica di un protocollo di rete.

> **Definizione 1** *(Protocollo di rete)* **.** Un **protocollo** di rete definisce il formato e l'ordine dei messaggi scambiati tra due o più entità in comunicazione, così come le azioni intrapese in fase di trasmissione e/o di ricezione di un messaggio o di un altro evento.

Internet, e le reti di calcolatori in generale, fanno un uso estensivo di protocolli. Si impiegano protocolli differenti per realizzare compiti diversi.

1.3. I componenti della rete

Precedentemente abbiamo visto che nelle reti di calcolatori, i calcolatori e gli atri dispositivi connessi a Internet sono detti **sistemi periferici** o **end system**, in quanto si trovano ai confini di Internet.
I sistemi periferici vengono anche detti **host** quando ospitano programmi applicativi quali browser, web browser o software di lettura e getsione della posta elettronica. Gli host, inoltre, vengono suddivisi in due categorie: **client** e **server**. I client sono host che richiedono servizi e tendono ad essere PC, laptop, smartphon e cosi via, mentre i server si occupano di erogare dei servizi e sono sostanzialmente macchine più potenti che memorizzano e distribuiscono pagine web o flussi video, ritrasmettono la posta elettronica e così via.

1.3.1. Le reti di acceso

Esaminiamo ora le **reti di access**, detti *access network*, cioè la rete che connette fisicamente un sistema al suo **edge router**, che è il primo router sul percorso dal sistema d'origine a un qualsiasi altro sistema di desetinazione collocato al di fuori della stessa rete di accesso.

1.4. Il nucleo della rete

Esaminati i confini di Internet, approfondiamo il nucleo della rete: una maglia di commutatori di pacchetti e collegamenti che interconnettono i sistemi periferici di Internet.

1.4.1. Commutazione di pacchetto

Le applicazioni distribuite scambiano **messaggi** che possono contenere qualsiasi cosa il progettista del protocollo desideri. Possono svolgere una funzione di controllo o contenere dati come in un messaggio di posta elettronica, un'immagine JPEG o un file audio. La sorgente suddivide i messaggi lunghi in parti più piccole note come **pacchetti**. Tra la sorgente e la destinazione, questi pacchetti viaggiano attraverso

collegamenti e **commutatori di pacchetto**. I pacchetti vengono trasmessi su ciascun collegamento a una velocità pari alla velocità totale di trasmissione del collegamento stesso.

1.4.2. Commutazione di circuito

Per spostare i dati in una rete di collegamenti e commutatori esistono due approcci fondamentali: la **commutazione di circuito** e la **commutazione di pacchetto**. Nelle reti a commutazione di circuito le risorse richieste lungo un perocrso per consentire la comunicazione tra sistemi periferici sono riservate per l'intera durata della sessione di comunicazione. Nelle reti a commutazioni di pacchetto, tali risorse non sono riservate; i messaggi di una sessione utilizzano le risorse quando necessario, e di conseguenza potrebbero dover attendere per accedere a un collegamento.
Le reti telefoniche sono esempi di reti a commutazione di circuito. Si consideri che cosa avviene quando una persona vuole inviare informaizoni a un altro soggetto. Prima che possa iniziare l'invio, la rete deve stabilire una connessione tra mittente e destinatario. Questo è un collegamento «a regola d'arte» in cui i commutatori sul percorso tra mittente e destinatario mantegono lo stato della connessione per tutta la durata della comunicazione. Nel gergo della telefonia questa connessione è detta **circuito**.

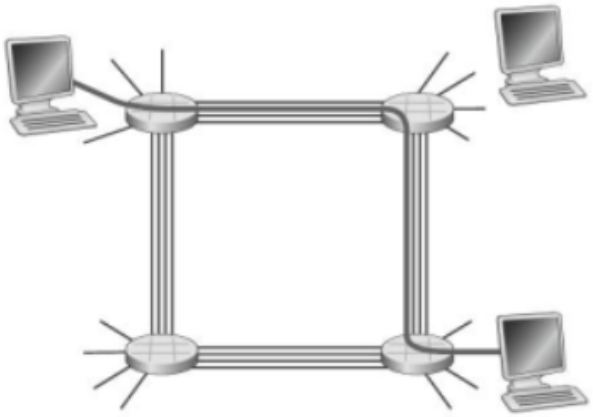
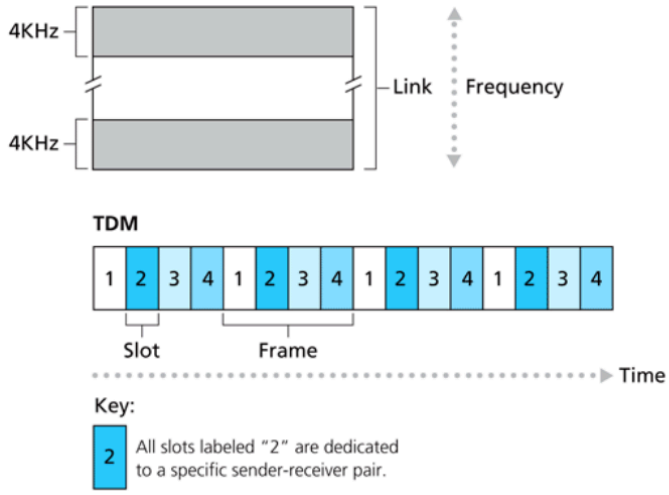


Figura 1: Rete a commutazione di circuito

La Figura 1 mostra una rete a commutazione di circuito in cui i quattro commutatori sono interconnessi tramite quattro colegamenti. Gli host sono tutti direttamente connessi a uno dei commutatori. Quando due host desiderano comunicare le rete stabilisce una **connessione end-to-end** dedicata a loro. Affinchè A invii messaggi a B, la rete deve prima riservare un circuito su ciascuno dei due collegamenti.
Si consideri invece che cosa accade quando un host invia un pacchetto a un altro host su una rete a commutazione di pacchetto, quale è Internet. Come nella commutazione a circuito, il pacchetto viene trasmesso su di una sequenza di collegamenti

ma, a differenza della commutazione di circuito, il pacchetto viene immesso nella rete senza che vengano riservate risorse. Se un collegamento è congestionato perchè vi sono altri pacchetti che devono essere trasmessi nello stesso istante, allora dovrà aspettare nel buffer del lato mittente e subire un ritardo.

Multiplexing nelle reti a commutazione di circuito
Un circuito all'interno di un collegamento è implementato tramite **multiplexing a divisione di frequenza (FDM, *frequency-division multiplexing*)** o **multiplexing a divisione di tempo (TDM, *time_division multiplexing*)**. Con FDM, lo spettro di frequenza di un collegamento viene suddiviso tra le connessioni stabilite tramite il collegamento. Nello specifico, il collegamento dedica una banda di frequenza a ciascun connessione per la durata della connessione stessa. La larghezza della banda viene detta **ampiezza di banda** o *bandwidth*.
Per un collegamento TDM il tempo viene suddiviso in frame di durata fissa, a lroo volta ripartiti in un numero fisso di slot temporali. Quando la rete stabilisce una connessione attraverso un collegamento, le dedica uno slot di tempo in ogni frame. Tali slot sono dedicati unicamente a quella connessione, con uno slot temporale disponibile in ciascun frame alla trasmissione dei dati di connessione.



La Figura 2 mnostra FDM e TDM per uno specifico collegamento di rete che supporta fino a 4 circuiti. Nel caso di FDM il dominio delle frequenze viene ripartito in quattro bande, ciascuna con ampiezza di 4 kHz. Nel caso di TDM, il dominio del tempo viene suddiviso in frame, con quattro slot di tempo per ciascun intervallo; a ogni circuito viene assegnato lo stesso slot dedicato in tutti i frame. Nel caso di TDM, la velocità di trasmissione di un circuito è uguale alla frequenza di frame moltiplicata per il numero di bit in uno slot.
Un'ulteriore strategia di multiplazione è la **WDM, Wavelength-Division Mul-**

tiplexing che viene usata per connessioni e fibra ottica e prevede una multiplazione e demultiplazione dei flussi che sfruttano diverse lunghezze d'onda della luce laser.

1.4.3. Una rete di reti

Abbiamo visto che i sistemi periferici si collegano a Internet tramite un ISP di accesso che può fornire connettività attraverso una rete cablata o senza fili. Tuttavia, la connessione degli utenti finali e dei fornitori di contenuti alla rete di un ISP è solouna piccola parte del puzzle per connettere i miliardi di utenti che costituiscono Internet. Per completare il puzzle bisogna interconnettere gli stessi ISP. Ciò avviene creando una **rete di reti**.
Per connettere gli ISP, un approccio ingenuo sarebbe quello di connettere direttamente ogni ISP di accesso con tutti gli altri, ma una struttura simile, detta a *maglia*, è troppo costosa.
Si adotta, quindi, una struttura gerarchica basata su livelli. Ci sono gli ISP di accesso di livello 3, gli ISP regionali di livello 2 e gli ISP globali di livello 1. Ogni ISP è cliente di uno del livello superiore e il cliente paga il fornitore per il transito verso il resto di Internet.
Un altra soluzione è quella di avere al centro poche decine di ISP di livello 1 e ciascuno è connesso a un gran numero di ISP di livello inferiore.
Il problema di questo tipio di connessione è il costo. Infatti gli ISP di livello inferiore pagano gli ISP di livello superiore e il costo riflette la quantità di traffico che l'ISP cliente scambia con il fornitore. Per ridurre tali costi, una coppia di ISP vicini e di pari livello gerarchico può fare uso di **peering**, cioè connettere direttametne le redi in modo che tutto il traffico tra di esse passi attraverso una connessione diretta. In questo modo due ISP si scambiano traffico senza pagarsi. L'**IXP**, Internet eXchange Point, è la sede fisica dove molte resi si interconnettono e fanno peering.

1.5. Tipi di reti

Esiste una grande varietà di tecnologie di rete e di modelli organizzativi. In particolare, le reti possono essere classificate in base all'estensione geografica:

- Rete personale, dette **PAN (Personal Area Network)**, sono reti in ambitoo personale e connettono apparati personali nel raggio di pochi metri: smartphone, cuffie, smartwatch, stampanti e così via. I collegamenti possono essere guidati tramite USB oppure via wireless;
- Rete locale, dette **LAN (Local Area Network)** e sono reti che si estendono all'interno di un edificio o di un comprensorio, con un'estensione nell'ordine del centinaio di metri. Le LAN hanno velocità trasmissive elevate (1-10 Gbps su cavo). Le LAN oggi sono reti commutate con collegamenti punto-punto e i collegamenti sono guidati e wireless;

- Rete metropolitana, dette **MAN (Metropolitan Area Network)** e sono reti in ambito cittadino. Il mezzo trasmissivo tipico è la fibra ottica sul backcon, oppure cavi in fibra/rame;
- Rete geografica, dette **WAN (Wide Area Network)** e sono reti in ambito nazionale o internazionale. I collegamenti sono indiretti guidati e wireless. Possono essere reti a commutazione di circuito e di pacchetto e le topologie possono essere svariate.

1.6. Ritardi, perdite e throughput nelle reti a commutazione di pacchetto

1.6.1. Ritardo nelle reti a commutazione di pacchetto

Un pacchetto parte da un host, passa attraverso una serie di router e conclude il viaggio in un altro host. A ogni tappa, il pacchetto subisce vari tipi di ritardo a ciascun nodo del tragitto. Di tali ritardi, i principali sono il **ritardo di elaborazione**, il **ritardo di accodamento**, il **ritardo di trasmissione** e il **ritardo di propagazione** che complessivamente formano il **ritardo totale di nodo**. Esaminiamoli nel dettaglio.

- il *ritardo di elaborazione* è il tempo richiesto per esaminare l'intestazione del pacchetto e per determinare dove dirigerlo. Questo può anche includere altri fattori, tra i quali il tempo richiesto per controllare errori a livello di bit.
- il *ritardo di accodamento* è dato dalla somma dei tempo di attesa per ogni coda
- il *ritardo di trasmissione* è dato dal tempo di immissione del pacchetto sul collegamento. È pari al rapporto delle dimensione del pacchetto (in bit) e la velocità di trasmissione (in bps).
- il *ritardo di propagazione* è dato dalla distanza/velocità del segnale sul mezzo trasmissivo.

Inoltre, ci sono altri tipi di ritardo, quali:

- **ritardo end-to-end** che è dato dalla somam dei ritardi impeigato da un pacchetto per andare da un nodo di partenza a quello di destinazione;
- **round trip time** (tempo di andata e ritorno) è il tempo impiegato da un pacchetto per andare da un punto all'altro della rete e tornare al unto di partenza ed è dato da $2 \cdot$ ritardo end-to-end

Per ottenere una misura efficiente dei ritardi utilizziamo due strumenti:

1. *ping* calcola il ritardo rispetto ad una destinazione finale per un messaggio di dimensioni standard;
2. *traceroute (tracert)* calcola il ritardo per ogni nodo intermedio fino alla destinazione finale per un messaggio di dimensioni standard.

Architetture a livelli

Precedentemente abbiamo descritto come è costituita la rete Internet, che è, quindi, un sistema complicato ed esiste un'organizzazione per organizzare l'architettura delle reti.
Un'architettura a livelli consente di discutere una parte specifica e ben definita di un sistema articolato e complesso. Questa semplificazione ha un valore importante grazie all'introduzione della modularità, che rende molto più facile cambiare l'implementazione del servizio fornito da un determinato livello. Fino a quando il livello fornisce lo stesso servizio allo strato superiore e utilizza gli stessi servizi dello strato inferiore, la parte rimanente del sistema rimane invariata al variare dell'implementazione del livello. In sistemi grandi e complessi, che vengono costantemente aggiornati, la capacità di cambiare l'implementazione di un servizio senza coinvolgere altre componenti del sistema costituisce un ulteriore importante vantaggio legato alla stratificazione.

2.1. Stratificazione dei protocolli

Per dare una struttura ai protocolli di rete, i progettisti organizzano protocolli e l'hardware che li implementano in **strati**, detti *layer*. Ciascun protocollo appartiene a uno dei livelli. Adesso rivolgiamo l'attenzione ai **servizi** offerti da un livello superiore: si tratta del cosidetto **modello di servizio**, detto *service model*, di un livello. Un livello di protocolli può essere implementato via software, hardware o con una combinazione dei due. I protocolli a livello di applicazione, quali HTTP e SMTP, sono quasi sempre implementati via software nei sistemi periferici; e così è anche per i protocolli a livello di trasporto. Dato che i livelli fisico e di data link si occupano della comunicazione su un collegamento specifico, sono di regola implementati nella scheda di rete associata. Il livello di rete è spesso un'implementazione mista di hardware e software.
La stratificazione dei protocolli presenta vantaggi concettuali e strutturali. Fornisce un modo strutturato per trattare i componenti dei sistemi. La modularità rende più facile aggiornare la componentistica. Uno svantaggio legato alla stratificazione è la possibilità che un livello duplichi le funzionalità di quello inferiore. Ad esempio, molte pile di protocolli forniscono meccanismi di correzione degli errori ai livelli sia di collegamento sia di connessione end-to-end. Un secondo potenziale svantaggio è che la funzionalità a un livello possa richiedere informazioni presenti solo in un altro livello e questo viola lo scopo insito nella separazione dei livelli.
Considerati assieme, i protocolli dei vari livelli sono detti **pila di protocolli**, detto

protocol stack. La pila di protocolli di Internet, mostrata nella seguente figura, consiste di cinque livelli: fisico, collegamento, rete, trasporto e applicazione.



Figura 3: Pila di protocolli di Internet

Analizziamo brevemente i vari livelli:

- il livello di *applicazione* è la sede delle applicazioni di rete e dei relativi protocolli. Per quanto riguarda Internet, questo livello include molti protocolli, come HTTP che consente la richiesta e il trasferimento dei documenti web, SMTP che consente il trasferimento dei messaggi di posta elettronica e FTP che consente il trasferimento di file tra due sistemi remoti. I pacchetti di informazione vengono chiamati **messaggi**.
- il livello di *trasporto* di Internet trasferisce i messaggi del livello di applicazione tra punti periferici gestiti dalle applicazioni. In Internet troviamo due protocolli di trasporto: TCP e UDP. TCP fornisce alle applicazioni un servizio orientato alla connessione, che include la consegna garantita dei messaggi a livello di applicazione alla destinazione e il controllo di flusso. Inoltre, TCP fraziona i messaggi lunghi in segmenti più piccoli e fornisce un meccanismo di controllo della congestione.
  Il protocollo UDP fornisce alle proprie applicazioni un servizio non orientato alla connessione che è un servizio senza affidabilità, né controllo di flusso e della congestione. I pacchetti a livello di trasporto si chiamano **segmenti**.
- Il lviello di rete di Internet si occupa di trasferire i pacchetti a livello di rete, detti **datagrammi**, da un host a un altro. Il livello di rete di Internet comprende il famoso protocollo IP, che definisce i campi dei datagrammi e come si sistemi periferici e i router agiscono su tali campi. Il livello di rete di Internet contiene svariati protocolli di instradamento che determinano i percorsi che i datagrammi devono seguire tra la sorgente e la destinazione.
- Il livello di *collegamento* si occupa di trasferire il datagramma al livello di rete superiore. I servizi forniti da questo livello dipendono dallo specifico protocollo utilizzato. Esempi di livello di collegamento includono Ethernet, Wi-fi e il proto-

collo di accesso alla rete DOCSIS. I pacchetti a livello di collegamento vengono chiamati **frame**.

- Il livello *fisico* è trasferire i singoli bit del frame da un nodo a quello successivo. Anche i protocolli di questo livello dipendono dal collegamento e in più dipendono dall'effettivo mezzo trasmissivo.

Un architettura a livelli alternativa al modello presentato prima, detto modello TCP/IP è il modello ISO/OSI, presentato nella seguente figura:



Figura 4: Modello ISO/OSI

ISO sta per International Organization for Standardization e OSI per Open Systems Interconnection. Il modello ISO/OSI è un sistema aperto con caratteristiche di interoperabilità. Esso è composto da 7 livelli:

1. il livello *fisico* che si occupa di trasmetter bit lungo un canale di comunicazione;
2. il livello di *collegamento* che si occupa di trasmettere i frame tra nodi adiacenti collegati direttamente, con rilevazione degli errori;

3. il livello di *rete* che si occupa di instradare i datagrammi tra reti a commutazione di pacchetto;
4. il livello di *trasporto* che si occupa di far comunicare, in modalità end-to-end, i processi trasmettendo segmenti;
5. il livello di *sessione* che si occupa di organizzare il dialogo di sincronizzare i programmi applicativi;
6. il livello di *presentazione* che si occupa di come rappresentare i dati da trasferire;
7. il livello di *applicazioni* che si occupa del trasferimento di messaggi tra le parti di una applicazione.

2.2. Incapsulamento

Quando un messaggio attraversa i vari livelli della rete si applica il cosiddetto meccanismo di **incapsulamento**. Presso un host mittente, un **messaggio a livello di applicazione**, *application-layer message*, viene passato a livello di trasporto. Questo livello prende il messaggio e gli concatena informazioni aggiuntine, dette informazioni di intestazione che saranno utilizzate dalla parte ricevente del livello di trasporto. Il messaggio a livello di applicazione e le informazioni di intestazione a livello di trasporto costituiscono il **segmento a livello di trasporto**, *transport-layer segment* che incapsula il messaggio a livello di applicazione. Le informazioni aggiunte potrebbero includere dati che consentono al livello di trasporto di consegnare il messaggio all'applicazione desiderata. Il livello di trasporto passa il segmento al livello di rete, che aggiunge informazioni di intestazioni proprie, quali gli indirizzi dei sistemi perfiferici di sorgente e di destinazione, creando un **datagramma a livello di rete**, *network-layer datagram*. A questo punto, il datagramma viene passato al livello di collegamento, il quale aggiunge le proprie informazioni di intestazioni creando un **frame a livello di collegamento**, *link-layer frame*. Quindi a ciascun livello, il pacchetto ha due tipi di campo: quello di intestazione e quello di **payload** (il carico utile trasportato). Il payload è tipicamente un pacchetto proveniente dal livello superiore.

Livello di applicazione

3.1. Web e HTTP

Fino agli anni "90 Internet veniva principalmente utilizzata da ricercatori, docenti e studenti universati per raggiungere host remoti, trasferire file, ricevere e inviare

notizie e per la posta elettronica. Poi, comparve il World Wide Web. Il Web è stata la prima applicazione Internet che ha cambiato profondamente il modo di interagire all"interno e all'esterno degli ambienti di lavoro.
Ciò che attira del Web è che opera su richiesta , si può avere ciò che si vuole, quando lo si vuole. Oltre a questo, il Web presenta molte altre interessanti caratteristiche. È facilissimo per tutti rendere disponibili informazioni sul Web: chiunque può diventare editore a costi estremamente bassi. Inoltre il Web fornisce una piattaforma a molte applicazioni killer, tra cui YouTube, servizi di posta elettronica come Gmail, Instagram e Google Maps.

3.1.1. Architettura del Browser e del Server

L'architettura di un web browser comprende diversi moduli specializzati: un Display manager, la gestione delle Estensioni/Plug-in, un Cache manager, un Request processor, il motore JavaScript (JS engine), un HTML interpreter e lo stack di rete HTTP/TCP.
Parallelamente, l'architettura di un web server (o **http daemon**) svolge le funzioni di gestire le connessioni con i browser e ritrovare o creare documenti dal File System. Include anche un modulo per l'interpretazione CGI/HTTP, un manager delle richieste e un Path Resolver per gestire le autorizzazioni di accesso. I web server oggi più diffusi sul mercato sono Apache, Nginx, Cloudflare e LiteSpeed, con IIS ormai marginale.

3.1.2. Differenza tra URI, URN e URL

Il sistema di indirizzamento del Web si basa formalmente sugli URI (Uniform Resource Identifier), definiti nella RFC 3986, che identificano una risorsa attraverso il nome o la posizione. Gli URI si dividono principalmente in due categorie:

- **URN (Uniform Resource Name):** Associa un identificatore univoco alla risorsa senza implicarne la disponibilità e senza specificare il meccanismo di accesso (es. un codice ISBN per un libro).
- **URL (Uniform Resource Locator):** Specifica sia il metodo di accesso sia la locazione fisica della risorsa, dandone per implicita la disponibilità; se la locazione cambia, il link si rompe («broken link»).

La sintassi generale di un URL è strutturata in questo modo: `scheme:["//"] [user [":"password] "@"] host [":"port] ["/"url-path]` . Lo **scheme** indica il protocollo (es. http, https, mailto, file), l'**host** è il nome DNS o l'indirizzo IP, la **port** è il numero di porta (opzionale, default 80 per HTTP o 443 per HTTPS) e l'**url-path** è il percorso gerarchico della risorsa seguito da eventuali parametri di query.

3.1.3. Panoramica di HTTP

HTTP, acronimo di *HyperText Transfer Protocol*, protocollo a livello di applicazione del Web, costituisce il cuore del Web. Questo protocollo è implementato in due

programmi, client e server, in esecuzione su sistemi periferici diversi che comunicano tra loro scamiandosi messaggi HTTP. Il protocollo definisce sia la struttura dei messaggi sia la modalità con cui client e server si scambiano i messaggi.
Una **pagina web**, detta anche documento, è costituita da oggetti. Un **oggetto** è semplicemente un file indirizzabile tramite un URL. La maggioranza delle pagine web consiste di un **file HTML principale** e diversi oggetti referenziati da esso. Ogni URL ha due componenti: il nome dell'host del server che ospita l'oggetto e il percorso dell'oggetto. Per esempio, l'URL

http.//www.someSchool.edu/someDepartment/picture.gif

ha *www.someSchool.edu* come nome dell'host e */someDepartment/picture.gif* come percorso. Un **browser web** implementa il lato client di HTTP. Un **web server**, che implementa il lato server di HTTP, ospita oggetti web, indirizzabili tramite URL.
HTTP definisce in che modo i client web richiedono le pagine ai web server e come questi ultimi le trasferiscono ai client. Quando l'utente richiede una pagina web, il browser invia al server messaggi di richiesta HTTP per gli oggetti nella pagina. Il server riceve le richieste e risponde con messaggi di risposta HTTP contenenti gli oggetti.
HTTP utilizza TCP (anzichè UDP) come protocollo di trasporto. Il client HTTP per prima cosa inizia una connessione TCP con il server. Una volta stabilita, i processi client e server accedono a TCP attraverso le proprie socket. L'interfaccia socket è la porta tra un processo e la sua connessione TCP. Il client invia richieste e riceve risposte HTTP tramite la propria interfaccia socket, analogamente il server riceve richieste e invia messaggi di risposta attraverso la propria interfaccia socket.
Quando il client ha mandato un messaggio alla sua interfaccia socket, questo non è più in suo possesso, ma si trova «nelle mani» di TCP. TCP mette a disposizione di HTTP un servizio di trasferimento dati affidabile. Questo è uno dei grandi vantaggi di un'architettura organizzata a livelli: HTTP non si deve preocuppare dei dati smarriti o di come TCP recuperi le perdite o riordini i dati all'interno della rete: questi sono compiti di TCP e dei protocolli di livello inferiore.
Il server invia i file richiesti ai clienti senza memorizzare alcuna informazione di stato a proposito dei client. Per cui, in caso di ulteriore richiesta dello stesso oggetto da parte dello stesso client, anche nel giro di pochi secondi, il server procedera nuovamente all'invio. Dato che i server HTTP non mantengono informazioni sui client, HTTP è classificato come **protocollo senza memoria di stato**. Un web server è sempre attivo, ha un indirizzo IP fisso e risponde potenzialmente alle richieste provenienti da milioni di diversi browser.

3.1.4. HTTP/1

La versione originale di HTTP si chiama HTTP / 1.0 e risale agli inizi degli anni "90. A partire dal 2020, la maggior parte delle transazioni HTTP avviene su HTTP/1.1.

Tuttavia, sempre più browser e web server supportano una nuova versione di HTTP chiamata HTTP/2.

3.1.5. Connessioni persistenti e non persistenti

In molte applicazioni per Internet, client e server comunicano per un lungo periodo di tempo, con il client che inoltra una serie di richieste e il server che risponde a ciascuna di esse. Quando tale interazione ha luogo su TCP, gli sviluppatori dell'applicazione devono prendere una decisione importante: devono decidere se ciascuna coppia richiesta/risposta deve essere inviata su TCP *separata* oppure devono essere inviate tutte sulla stessa connessione TCP. Nel primo approccio si dice che l'applicazione usa **connessioni non persistenti**, mentre nel secondo usa **connessioni persistenti**.

HTTP con connessioni non persistenti

Analizziamo il trasferimento di un file nel caso di connessione non persistente. Supponiamo che la pagina consista di un file HTML principale e di 10 immagini JPEG, e che tutti gli undici oggetti risiedano sullo stesso server. Ipotezziamo che l'URL del file HTML principale sia:

http://wwww.someSchool.edu/someDepartment/home.index

Le fasi della connessione sono le seguenti.

1. Il processo client HTTP inizializza una connessione TCP con il server *www.someSchool.edu* sulla porta 80, che è la porta di default per HTTP. Associate alla connessione TCP ci saranno una socket per il client e una per il server.
2. Il client HTTP, tramite la propria socket, invia al server un messaggio di richiesta HTTP che include il percorso */someDeparment/home.index*.
3. Il processo server HTTP riceve il messaggio di richiesta attraverso la propria socket associata alla connessione, recupera l'oggetto */someDeparment/home.index* dalla memoria, lo incapsula in un messaggio di risposta HTTP che viene inviato al client attraverso la socket.
4. Il processo server HTTP comunica a TCP di chiudere la connessione. Questo, però, non termina la connessione finchè non sia certo che il client abbia ricevuto integro il messaggio di risposta.
5. Il client HTTP riceve il messaggio di risposta. La connessione TCP termina. Il messaggio indica che l'oggetto incapsulato è un file HTML. Il client estrae il file dal messaggio di risposta, esamina il file HTML e trova i riferimenti ai 10 oggetti JPEG.
6. Vengono quindi ripetuti i primi 4 passi per ciascuno degli oggetti JPEG referenziati.Questi passi illustrano l'utilizzo di connessioni non persistenti in cui ogni connessione TCP viene chiusa dopo l'invio dell'oggetto da parte del

server: vale a dire che ciascuna trasporta soltanto un messaggio di richiesta e un messaggio di risposta.

Possiamo fare un calcolo approssimativo per stimare l'intervallo di tempo che intercorre tra la richiesta di un file HTML da parte del client e il momento in cui l'intero file viene ricevuto dal client/ A questo scopo, definiamo il **round-trip time (RTT)**, che rappresenta il tempo impiegato da un singolo pacchetto per viaggiare dal client al server e poi tornare al client. RTT include i ritardi di propagazione, di accodamento nei router e nei commutatori intermedi nonchè di elaborazione del pacchetto. Quando l'utente clicca su un collegamento ipertestuale, invece, questo comporta un **handshake a tre vie**: il client invia un piccolo segment TCP al server e quest'ultimo manda una conferma per mezzo di un piccolo segmento TCP. Infine il client dà anch'esso una conferma di ritorno al server. Le prime due parti dell'handshare a tre vie richiedono un RTT. Dopo il loro completamento, il client invia un messaggio di richiesta HTTP combinano con la terza parte dell'handshake, la conferma di avvenuta ricezione, tramite la connessione TCP>Quando il messaggio di richiesta arriva al server, quest'ultimo inoltra il file HTML sulla connessione TCP. La richiesta-risposta HTTP consuma un altro RTT. Pertanto, il tempo di risposta è approssivamente di due RTT più il tempo di trasmissione da parte del server del file HTML.

HTTP con connessioni persistenti

Le connessioni non persistenti presentano alcuni limiti:

1. per ogni oggetto richiesto occorre stabilire e mantenere una nuova connessione. Per ogni connessione si devono allocare buffer e mantenere variabili TCP sia nel client sia nel server.
2. ciascun oggetto subisce un ritardo di consegna di due RTT, uno per stabilire la connessione TCP e uno per richiedere e ricevere un oggetto.

Con HTTP 1.1 nelle connessioni persistenti il server lascia la connessione TCP aperta dopo l'invio di una risposta, per cui le richieste e le risposte successive tra gli stessi client e server possono essere trasmesse sulla stessa connessione. Inoltre, il server può inviare un'intera pagina web o anche più pagine web allo setsso client.
Queste richieste di oggetti possono essere effettuate una di seguito all'altra senza aspettare le risposte delle richieste pendenti (meccanismo di *pipeling*). In generale, il server HTTP chiude la connessione quando essa rimane inattiva per un dato lasso di tempo. La modalità di default di HTTP impiega connessioni persistenti con pipeling.

3.1.6. Formato dei messaggi HTTP

Le specifiche HTTP includono la definizione dei due formati dei messaggi HTTP, di richiesta e di risposta.

Messaggio di richiesta HTTP

Un messaggio tipico di richiesta HTTP si presenta nel seguente modo:

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

Il messaggio è scritto in testo ASCII, in modo che l'utente sia in grado di leggerlo. Inoltre, consiste di cinque righe, ciascuna seguito da un carattere di ritorno a capo e un carattere di nuova linea aggiuntivi. In generale, i messaggi di richiesta possono essere costituito da un numero indefinito di righe, anche una sola. La prima riga è detta **riga di richiesta** (*request line*), quelle successive **righe di intestazione** (*header lines*). La riga di richiesta presenta tre campi: il campo metodo, il campo URL e il campo versione di HTTP. Il campo metodo può assumere diversi valori, tra cui `GET, POST, HEAD, PUT` e `DELETE`. La maggioranza dei messaggi di richiesta HTTP usa il metodo `GET`, adottato quando il browser richiede un oggetto identificato dal campo URL. La versione è auto esplicativa.
Consideriamo le righe di intestazione dell'esempio. La riga *Host: www.someSchool.edu* specifica l'host su cui risiede l'oggetto. Includendo la linea di intestazione *Connection: close*, il browser sta comunicando al server che non si deve occupare di connessioni persistenti, ma vuole che questi chiuda la connessione dopo aver inviato l'oggetto richiesto. La riga di intestazione *User-agent:* specifica il tipo di browser che sta effettuando la richiesta al server. Questa riga è utile in quanto il server può inviare versioni diverse dello stesso oggetto a browser di tipi diversi. Infine, *Accept-language:* indica che l'utente preferisce ricevere una versione in francese dell'oggetto se disponibile; altrimenti, il server dovrebbe inviare la versione di default.
Dopo le linee di intestazione si trova un «corpo». Nel caso di metodo GET è vuoto, ma viene utilizzato nel metodo POST, il quale viene usato per inviare dati al server. Le richieste generate con il form non devono necessariamente usare il metodo POST. Anzi, spesso i form HTML utilizzano il metodo GET E includono i dati immessi nell'URL richiesto. Ad esempio:

www.somesite.com/animalsearch?scimmie&banane

Il metodo HEAD è simile a GET. Quando un server riceve una richiesta con il metodo HEAD, risponde con un messaggio HTTP, ma tralascia gli oggetti richiesti. Gli sviluppatori spesso utilizzano il metodo HEAD per verificare la correttezza del codice prodotto. Il metodo PUT, consente agli utenti di inviare un oggetto a un percorso specifico su uno specifico web server.

Messaggio di risposta HTTP

Analizziamo un tipico messaggio di risposta HTTP che potrebbe rappresentare la risposta al messaggio di richiesta dell'esempio precedente.

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
(data data data data data ...)
```

Analizzando questo messaggio di risposta ci sono tre sezioni:

1. una **riga di stato** iniziale. Questa presenta tre campi:
   1. la versione del protocollo
   2. un codice di stato
   3. un corrispettivo messaggio di stato;
2. sei **righe di intestazione**. Il server utilizza la riga di intestazione *Connection: close* per comunicare al client che ha intenzione di chiudere la connessione TCP dopo l'invio del messaggio. La riga *Date:* indica l'ora e la data di creazione e invio, da parte del server, della riposta HTTP. La riga *Server:* indica che il messaggio è stato generato da un web server Apache. La riga *Last-Modified*: indica l'istante e la data in cui l'oggetto è stato creato o modificato per l'ultima volta. La riga di intestazione *Content-Length:* contiene il numero di byte dell'oggetto inviato. La riga *Content-Type* indica che l'oggetto nel corpo è testo HTML. ;
3. il **corpo**: questo è il fulcro del messaggio contiene l'oggetto richiesto.

Analizziamo alcuni codici di stato e le relative espressioni.

1. 200 OK: la richiesta ha avuto successo e in risposta si invia l'informazione;
2. 301 Moved Permanently: l'oggetto richiesto è stato trasferito in modo permanente; il nuovo URL è specificato nell'intestazione *Localtion:* del messaggio di risposta.
3. 400 Bad Request: si tratta di un codcie di errore generico che indica che la richiesta non è compresa dal server;
4. 404 Not Found: il documento richiesto non esiste sul server.
5. 505 HTTP Version Not Supported: il server non dispone della versione di protocollo HTTP richiesta.

Il campo `Content-Type` sfrutta lo standard MIME (Multipurpose Internet Mail Extensions) per indicare la natura dei dati trasmessi. Tra i tipi e sottotipi più comuni troviamo testo (`text/plain` , `text/html` , `text/xml`), immagini (`image/gif` , `image/jpeg`), audio (`audio/basic`), video (`video/mpeg`) e file applicativi (`application/pdf`).

Per quanto riguarda i codici di stato, il protocollo HTTP li suddivide in grandi categorie:

- 2xx (Successful): Include 200 OK, 201 Created (utilizzato per POST/PUT) e 204 No Content.
- 3xx (Redirection): Include 301 Moved Permanently, 302 Found (trasferimento temporaneo) e 304 Not Modified (indica che la cache è ancora valida).
- 4xx (Client Error): Include 400 Bad Request, 401 Unauthorized, 403 Forbidden e 404 Not Found.
- 5xx (Server Error): Include 500 Internal Server Error, 501 Not Implemented, 502 Bad Gateway, 503 Service Unavailable e 504 Gateway Timeout.

Inoltre, per velocizzare il caricamento con connessioni persistenti (HTTP/1.1), va notato che i browser moderni adottano un approccio fuori dallo standard originario (che suggeriva max 2 connessioni), aprendo in parallelo fino a 6 connessioni TCP per ciascun hostname.

3.1.7. Interazione utente-server: i cookie

Abbiamo detto che i server HTTP sono privi di stato. Ciò semplifica la progettazione e consente di sviluppare web server ad alte prestazioni, in grado di gestire migliaia di connessioni TCP simultanee. Spesso i web server possono autenticare gli utenti, sia per limitare l'accesso da parte di questi ultimi sia per fornire contenuti in funzione della loro identità. A questo scopo HTTP adotta i **cookie**. I cooie, consentono ai server di tener traccia degli utenti. La tecnologia dei cookie presenta quattro componenti:

1. una riga di intestazione nel messaggio di risposta HTTP;
2. una riga di intestazione nel messaggio di richiesta HTTP;
3. un file mantenuto sil sistema dell'utente e gestito dal browser;
4. un database sul sito.

Quando un utente visita per la prima volta un sito, il sito crea un identificativo unico e una voce nel proprio database, indicizzata dal numero identificativo. A questo punto il server risponde, includendo nella risposta HTTP l'intestazione *Set-cookie:* che contiene il numero identificativo. Quando il browser del client riceve il messaggio di risposta HTTP, vede l'intestazione *Set-cookie* e aggiunge una riga al file dei cookie che gestisce. Questa riga include il nome dell'host del server e il numero identificativo. Quando l'utente visita nuovamente lo stesso sito, il suo browser consulta il suo file di cookie, estrae il suo numero identificativo per il sito e pone nella richiesta HTTP una riga di intestazione del cookie che include tale numero. Aggiunge la riga di intestazione *Cookie:* seguita dal numero identificativo. I cookie, quindi, possono essere usati per identificare gli utenti. La prima volta che visita un sito, un utente può fornire un'identificazione. Successivamente il browser passa un'intestazione di cookie al server durante tutte le successive visite al sito,

identificando l'utente sul server.
Le informazioni dei cookie sono memorizzate lato client dal browser in uno spazio dedicato (il «cookie store»), che oggigiorno è tipicamente implementato come un database SQLite. Non dovrebbero mai essere utilizzati per memorizzare password o altre informazioni sensibili in chiaro. Quando il server risponde con un'intestazione `Set-Cookie`, può specificare una serie di attributi predefiniti per regolarne il ciclo di vita e la sicurezza, come `Expires`, `Max-Age` (scadenza), `Domain`, `Path`, `Secure` e `HttpOnly`.

3.1.8. Web caching

Una **web cache**, nota anche come **proxy server**, è un'entità di rete che soddisfa richieste HTTP al posto del web server effettivo. Il proxy ha una propria memoria sul disco (una cache) in cui conserva copie di oggetti recentemente richiesti. Il browser di un utente può essere configurato in modo che tutte le richieste HTTP dell'utente vengano innanzitutto dirette al proxy server. Una volta configurato il browser, ogni richiesta di oggetto da parte del browser viene diretta al proxy. Quando un browser richiede un oggetto, accade che:

1. Il browser stabilisce una connessione TCP con il proxy server e invia una richiesta HTTP per l'oggetto specificato.
2. Il proxy controlla la presenza di una copia dell'oggetto memorizzata localmente. Se l'oggetto viene rilevato, il proxy lo inoltra all'interno di un messaggio di risposta HTTP al browser.
3. Se, invece, la cache non dispone dell'oggetto, il proxy apre una connessione TCP verso il server di origine. Quindi, il proxy invia al server una richiesta HTTP per l'oggetto. Una volta ricevuta tale richiesta, il server di origine invia al proxy l'oggetto all'interno di una risposta HTTP.
4. Quando il proxy riceve l'oggetto, ne salva una copia nella propria memoria locale e ne inoltra un'altra copia, all'interno di messaggio di risposta HTTP, al browser.

Il proxy è contemporaneamente server e client: quando riceve richiesta da un browser e gli invia risposte agisce da server, quando invia richieste e riceve risposte da un server di origine funziona da client.
Il web caching si è sviluppato in Internet per due ragioni. Innanzitutto, un proxy può ridurre in modo sostanziale i tempo di risposta alle richieste dei client, in particolare se l'ampiezza di banda che costituisce il collo di bottiglia tra il client e il server di origine è molto inferiore rispetto all'ampiezza di banda minima tra client e proxy. In secondo luogo, i proxy possono ridurre il traffico sul collegamento di accesso a Internet, con il vantaggio di non dover aumentare l'ampiezza di banda frequentemente e ottenere quindi una riduzione dei costi.
Un'altra alternativa al proxy server è il **reverse proxy server** che è un proxy che sta davanti ai server di origine. Un client manda la richiesta al proxy credendo che sia il web server e il proxy la inoltra a uno dei server di orogine, che restano nascosti

al client. Il reverse proxy server è il simmetrico del forward proxy: quello agisce per conto dei client, questo per conto dei server. Il reverse proxy server è importante per la sicurezza perchè nasconde l'indirizzo IP dei server, è importante per il load balancing e per le funzionalità di caching.

3.1.9. HTTP/2

La maggior parte dei browser, supporta HTTP/2. Gli obiettivi principali di HTTP/2 sono quello di ridurre la latenza percepita attivando il multiplexing di richiesta e risposta su singola connessione TCP, di fornire supporto per la priorità delle richieste e il server push e di fornire una compressione efficiente dei campi di intestazione HTTP. HTTP/2 non modifica metodi, codici di stato, URL e campi di intestazione di HTTP, ma cambia il modo in cui i dati vengono formattati e trasportati tra il client e il server. Abbiamo detto che HTTP/1.1 utilizza connessioni TCP persistenti ma l'invio di tutti gli oggetti web su una singola connessione web ha il problema dell'**HEAD OF LINE (HOL) blocking**. Per comprendere il blocco HOL, si consideri una pagina web che include una pagina di base HTML, un grande video clip e molti piccoli oggetti sotto il video. Supponiamo inoltre che ci sia un collegamento a collo di bottiglia con velocità medio-bassa sul percorso tra server e client. Utilizzando una singola connessione TCP, il video clip impiegherebbe molto tempo per passare attraverso il collegamento a collo di bottiglia e gli oggetti piccolo subirebbero un ritardo.
Per aggirare questo problema è possibile aprire più connessioni TCP parallele per trasportare una singola pagina web, «imbrogliando» e prendersi una parte maggiore della larghezza di banda del collegamento. Uno degli obiettivi principali di HTTP/2 è eliminare o ridurre il numero di connessioni TCP parallele per il trasporto di una singola pagina web.
Per implementare il multiplexing e aggirare il blocco HOL, HTTP/2 utilizza il «framing», suddividendo la risposta in frame binari di dimensione limitata (la dimensione massima è negoziata, con un default di 16 KB). Questo permette un «interleaving» (intervallamento) fluido dei frame di oggetti diversi. HTTP/2 (basato in origine sul protocollo SPDY di Google) fornisce anche supporto per la priorità: il client può indicare per ogni richiesta un livello di urgenza da 0 a 7 (RFC 9218). Inizialmente era previsto anche il **server push** (l'invio non richiesto di asset per la pagina), ma questa funzionalità è stata di fatto abbandonata e rimossa dai browser basati su Chromium a partire dal 2022.

3.1.10. HTTP/3

QUIC è un nuovo protocollo di trasporto implementato nel livello di applicazione sul portocollo UDP. QUIC offre diverse opportune funzionalità per HTTP, come il multiplexing dei messaggi (*interleaving*), il controllo di flusso per ogni stream e la creazione di connessioni a bassa latenza. HTTP/3 è un altro, nuovo protocollo

HTTP progettato per funzionare su QUIC. HTTP/3 non è ancora stato completmente standardizzato. Molte delle funzionalità HTTP/2 sono incluse in QUIC, facilitando la progettazione di HTTP/3.
HTTP/3 implementa in modo nativo la sicurezza: integra infatti il protocollo TLS (Transport Layer Security), ovvero lo stesso protocollo alla base delle connessioni HTTPS, garantendo comunicazioni sicure di default.

3.1.11. Streaming video e reti per la distribuzione di contenuti

Lo streaming video rappresenta circa l'80% del traffico Internet (statistica del 2020).Analizziamo come i servizi streaming vengano implementati nell'Internet di oggi.

Video su Internet

Nelle applicazioni di streaming di video registrati, i contenuti sono video, memorizzati su server a disposizione degli utenti su richiesta. Innanzitutto, analizziamo le caratteristiche dei video. Un video è una sequenza di immagini, visualizzate a velocità costante di, ad esempio, 24 o 30 immagini al secondo. Un'immagine non compressa e codificata digitalmente consiste di un array di pixel, ognuno dei quali codificato con un numero di bit per rappresentare luminanza e crominanza. I video possono essere compressi in modo da raggiungere un compromesso tra qualità del video e bit rate. Gli algoritmi di compressione oggi disponibili sono in grado di comprimere un video a qualsiasi bit rate si desideri. Più è alto il bit rate, migliore è la qualità dell'immagine e l'esperienza visiva globale dell'utente.
Dal punto di vista del networking, la caratteristica più importante è l'elevata velocità con cui è necessario inviare i bit sulla rete (**bit rate**). La misura in assoluto più importante per lo streaming video è il throughput medio, il cui valore deve essere almeno pari a quello del bit rate del video per avere una riproduzione continua.
La compressione può essere usata anche per creare versioni multiple dello stesso video, a livello di qualità diversi. Ad esempio, si può usare la compressione per creare tre versioni dello stesso video, a 300 kbps, 1 Mbps e 3 Mbps e gli utenti possono decidere quale versione guardare in funzione della larghezza di banda disponibile.

Streaming HTTP e DASH

Nello streaming HTTP il video viene semplicemente memorizzato in un server HTTP come un file ordinario con un URL specifico. Quando un utente vuole vedere un video, il client stabilisce una connessione TCP con il server e invia una richiesta GET HTTP per il suo URL. Il server invia il file video, all'interno di un messaggio di risposta HTTP, più velocemente possibile dati il traffico e i protocolli di rete. Sul lato client i byte vengono memorizzati in un buffer dell'applicazione client. Quando il numero di byte nel buffer supera una soglia fissata, l'applicazione client inizia la riproduzione: periodicamente prende i frame video dal buffer, li decomprime e li visualizza all'utente. Quindi l'applicazione di video streaming consiste nel visua-

lizzare i frame mentre li riceve, memorizzando nel buffer gli ultimi.
Lo streaming HTTP presenta un ande svantaggio: i client ricevono tutti la stessa versione codificata del video, nonstante abbiano a disposizione una larghezza di banda che può variare sensibilmente da un caso all'altro e nel tempo. Per superare questo problema è stato sviluppato un nuovo tipo di streaming basato su HTTP, chiamato **streaming dinamico adattativo su HTTP**, *Dynamic Adaptive Streming Over HTTP*, detto **DASH**. In DASH, i video vengono codificati in diverse versioni, ognuna avente un bit rate differente e quindi un differente livello di qualità. Il client richiede pezzi di segmenti video lunghi alcuni secondi da versioni differenti in modo dinamico. Quando la banda disponibile è elevata, il client seleziona automaticamente i blocchi da una versione con alto bit rate, mentre quando la banda disponibile è pcoa, seleziona una versione a basso bit rate. Il client seleziona un blocco alla volta con un messaggio di richiesta GET HTTP.
Da una parte, DASH permette a client con accessi a Internet diversi di fare streaming del video con codifiche diverse: client con connessioni lente possono ricevere versioin a basso bit rate e quindi a bassa qualtià, mentre client con connessioni in fibra possono ricevere una versione ad alta qualità. D'altra canto, DASH permette anche a un client di adattare la scelta della versione alla banda disponibile se varia durante la sessione. Tale caratteristica è importante per gli utenti mobili, per i quali la banda disponibile fluttua quando cambiano stazione base.
Con DASH, i video nelle varie versioin sono memorizzati nel server HTTP, ognuno a un URL diverso. Il server HTTP ha anche un file di descrizione, detto **manifest file**, che per ogni versione fornisce il rispettivo URL insieme al bit rate. Il client richiede questo file per venire a conoscenza delle varie versioni disponibili. Quindi seleziona a ogni istante un blocco, specificando nel messaggio di richiesta GET di HTTP un URL e un intervallo di byte. Mentre scarica i blocchi, il client misura la banda di ricezione ed esegue un algoritmo per selezionare il blocco successivo. Naturalmente il client, se ha molti dati del video memorizzati nel buffer e la banda di ricezione misurata è large, sceglie una versione ad alto bit rate per il blocco successivo; al contrario, se ha pochi dati nel buffer e banda bassa, sceglie una versione basso rate. Pertanto DASH permette al client di cambiare liberamente il livello di qualità.

Reti per la distribuzione di contenuti

Oggigiorno molte aziende di video in Internet distribuiscono quotidianamente video on demand con streaming dell'ordine dei Mbps a milioni di utenti. Mandare in streaming una tale mole di traffico a utenti sparsi in tutot il mondo, fornendo riproduzione continua e alta interattività è complesso. L'approccio più diretto sarebbe costruire un unico enorme data center, memorizzare in esso tutti i video e mandarli in streaming direttamente. Questo approccio però presenta tre grandi problemi:

1. Se il client è lontano dal data center, i pacchetti dal server al client devono percorrere un lungo cammino passando per molti ISP, magari in continenti diversi.

Se uno dei collegamenti ha un throughput minore della velocità di consumo del video, anche il throughput totale end-to-end lo sarà, causando continui e fastidiosi fermi immagine.

2. Un video molto popolare verrebbe inviato molte volte sullo stesso collegamento, non solo sprecando banda, ma constrigendo anche l'azienda a pagare tutte le volte il suo ISP per inviare gli stessi dati.
3. Un singolo data center rappresenta un singolo punto di rottura: se il data center o il collegamento da esso a Internet si interrompesse, l'azienda non sarebbe più in grado di distribuire alcun video.

Per superare questi problemi, quasi tutte le maggiori aziende di video streaming usano le **CDN** (*Content Distribution Networks*). Una CDN gestisce server distribuiti in molti posti diversi, memorizza copie dei video e di altri contenuti web nei server e cerca di dirigere le richieste degli utenti al punto della CDN in grado di offrire il servizio migliore. La CDN può usare una **CDN privata**, cioè proprietaria del fornitore di contenuti. Alternativamente può essere usata la **CDN di terze parti** che distribuisce contenuti per conto di molti fornitori di contenuti. Le CDN generalmente adottano una delle due politiche di dislocazione dei server che seguono:

- **Enter deep** («entrare in profondità»). Una filosofia è quella di entrare in profondità nelle reti di accesso degli ISP installando gruppi di server, detti anche cluster, negli ISP di accesso sparsi in tutto il mondo. Questo approccio viene usato distribuendo server in migliaia di posti diversi. L'obiettivo è quello di essere vicini agli utenti finali in modo da migliorare il ritardo percepito dall'utente e il throughput, diminuendo il numero di collegamenti e router tra l'utente finale e il cluster CDN da cui riceve i contenuti. Questo approccio pone però grandi sfide nella manuntenzione e gestione dei cluster.
- **Bring home** («portare a casa»). Il secondo approccio è quello di portarsi in caso l'ISP costruendo grandi cluster in pochi punti chiave e interoconneterli usando una rete privata ad alta velocità. invece di entrare negli ISP di accesso, queste CDN pongono i loro cluster negli IXP. Tale approccio presenta meno problemi di gestione e manutenzione, ma spesso anche una minore qualità di servizio.

Una volta posizionati i cluster, la CDN replica i contenuti su di essi. I video raramente richiesti non vengono copiati su tutti i cluster.
Molte CDN non spingono i loro video verso i cluster, ma usano una semplice strategia per cui se un client richiede un video a un cluster che non lo ha memorizzato, il cluster recupera il video da un archivio centrale o da un altro cluster e ne memorrizza localmente una copia mentre lo manda in streaming al client.

Come funziona la CDN

Quando un browser nell'host di un utente chiede il recupero di uno specifico video identificato da un URL, la CDN deve interecettare la richiesta in modo d poter

1. determinare il cluster di server più appropriato per quel client a quell'istante
2. dirigere la richiesta del client a uno dei server di quel cluster.

Esaminiamo i meccanismi che stanno dietro l'intercettazione e il reindirizzamento di una richiesta, discutendo poi come una CDN decida quale sia il cluster più adatto. Molte CDN sfruttano il servizio DNS per intercettare e ridirigere le richieste. Consideriamo un esempio dove un fornitore di contenuti, *NetCinema*, impieghi una CDN di terza parte, *KingCDN*, per distribuire i suoi video. A ogni video viene assegnato un URL che include la stringa «video» e un identificatore univoco per tale video. Vengono compiuti 6 passi:

1. L'utente visita la pagina web di *NetCinema*;
2. Quando l'utente seleziona il link *http://video.netcinema.com/6Y7B23V* , il suo host invia un'interrogazione DNS a *video.netcinema.com*
3. Il **DNS server locale (LDNS)** dell'utente invia l'interrogazione DNS a un DNS server autoritativo per *NetCinema* che osserva la stringa «video» nel nome dell'host *video.netcinema.com*. Il DNS server autoritativo per *NetCinema* per passare l'interrogazione DNS a *KingCDN*, invece di restituire un indirizzo IP, restituisce all'LDNS il nome di un host nel dominio di *KingCDN*.
4. Da questo momento l'interrogazione DNS entra nell'infrastruttura DNS privata di *KingCDN*. Lo LDNS dell'utente invia una seconda interrogazione e infine il sistema DNS di *KingCDN* restituisce gli indirizzi IP di un server *KingCDN* all'LDNS. È quindi qui, all'interno del sistema DNS di *KingCDN*, che viene specificato il server CDN da cui il client riceverà il contenuto.
5. LDNS inoltra l'indirizzo IP del nodo CDN che fornirà il contenuto all'host dell'utente.
6. Il client, una volta ricevuto l'indirizzo IP del server di *KingCDN*, stabilisce una connessione TCP diretta con il server a quell'indirizzo IP e gli invia una richiesta GET HTTP per il video. Nel caso venga impiegato DASH, il server innanzitutto inverià al client il file manifesto con una lista di URL.

Strategie di selezione dei cluster

Il cuore dell'installazione di una CDN è la **strategia di selezione dei cluster**: un meccanismo per dirigere dinamicamente i client a un cluster di server o a un data center della CDN. La CDN apprende l'indirizzo IP del server LDNS del client attraverso la richiesta DNS del client. Dopo aver appreso tale indirizzo IP, la CDN deve selezionare un cluster appropriato. Le strategie di selezione del cluster sono generalmente proprietarie.

Una semplice strategia consiste nell'assegnare a un client il cluster *geograficamente più vicino*. Usando un database commerciale di geo-localizzazione, gli indirzzi IP degli LDNS vengono associati a un luogo geografico. La CDN, quando riceve una richiesta DNS da un particolare LDNS, sceglie il cluster a esso più vicino geograficamente in linea d'aria. Tale soluzione potrebbe non andare bene per alcuni client,

in quanto il cluster più vicino potrebbe essere diverso da quello più vicino dal punto di vista del percorso in rete. Un altro problema derivato da tutti gli approcci basati sul DNS deriva dal fatto che alcunu utenti sono configurati per usare LDNS remoti. Inoltre questa strategia non tiene conto delle variazioni temporali del ritardo di rete e della banda disponibile.
Un secondo approccio determina il cluster migliore per un client basandosi sulle condizioni di traffico correnti, effettuando **misure in tempo reale** delle prestazioni di ritardo e perdita tra cluster e client. Un problema di questo approccio è che molti LDNS sono configurati per non rispondere a tali richieste.

3.2. DNS: il servizio directory di Internet

Gli host Internet possono essere identificati in vari modi. Gli **hostname** quali *www.facebook.com*, *www.google.com*, risultano abbastanza approprati per le persone, ma forniscono poca informazione sulla loro collocazione all'interno di Internet.
Un nome quale *www.eurecom.fr*, che termina con il suffisso del paese, *.fr*, ci dice che l'host si trova probabilmente in Francia, ma niente di più. Inoltre, dato che questi nomi sono costituiti da un numero variabile di caratteri alfanumerici, sarebbero difficilmente elaborabili dai server. Per tali motivi, gli host vengono identificati dai cosiddetti **indirizzi IP**.
Un indirizzo IP consiste di quattro byte e presenta una rigida struttura gerarchica. Ha una forma del tipo 121.7.106.83, in cui ogni punto separa uno dei byte espressi con un numero decimale compreso tra 0 e 255. Un indirizzo IP è gerarchico perchè, leggendolo da sinistra a destra, otteniamo informazioni semper più specifiche sulla collocazione dell'host in Internet.

3.2.1. Servizi forniti da DNS

Quindi, esistono due modi per identificare gli host: il nome e l'indirizzo IP. Le persone preferiscono il primo, mentre i router prediligono gli indirizzi IP a lunghezza fissa e strutturati in modo gerarchico. Al fine di conciliare i due approcci è necessario un servizio che traduce gli hostname nei loro indirizzi IP (o viceversa). Si tratta del principale compito del *domain name system*, **DNS**. DNS è

1. un database distribuito implementato in una gerarchia di **DNS server**
2. un protocollo a livello di applicazione che consente agli host di interrogare il database.

I DNS server sono generalmente macchine UNIX che eseguono un software chiamato BIND, Berkley Internet Name Domain. Il protocollo DNS utilizza UDP e la porta 53 per mantenere una bassa latenza. Il protocollo DNS utilizza, invece, il protocollo TCP sulla porta 53 quando ci sono molti dati, che superano i 512 byte, da trasferire in modo affidabile. L'estensione EDNS permette oggi di inviare risposte UDP più grandi, fino a 1232 byte, limitando la necessità di passare a TCP.

DNS viene utilizzato da altri protocolli a livello di applicazione, tra cui HTTP e SMTP, per tradurre i nomid i host forniti dall'utente in indirizzi IP.
Analizziamo cosa succede quando un browser in esecuzione sull'host di un utente richiede l'URL *www.someschool.edu/index.html*. L'host dell'utente, per essere in grado di inviare un messaggio di richiesta HTTP al web server *www.someschool.edu*, deve ottenere prima il suo indirizzo IP. Ciò avviene come segue.

1. La stessa macchina utente esegue il lato client dell'applicazione DNS.
2. Il browser estrae il nome dell'host, *www.someschool.edu*, dall'URL e lo passa al lato client dell'applicazione DNS.
3. Il client DNS invia un'interrogazione contenente l'hostname a un DNS server.
4. Il client DNS prima o poi riceve una risposta, che include l'indirizzo IP corrispondente all'hostname.
5. Una volta ricevuto l'indirizzo IP dal DNS, il browser può dare inizio a una connessione TCP verso il processo server HTTP collegato alla porta 80 di quell'indirizzo IP.

Il DNS introduce un ritardo aggiuntivo, talvolta sostanziale, alle applicazioni Internet che lo utilizzano. Per evitare ciò, l'indirizzo IP desiderato si trova spesso nella cache di un DNS server vicino, il che aiuta a ridurre il traffico DNS in rete e il ritardo medio del servizio.
Oltre alla traduzione degli hostname in indirizzi IP, DNS mette a disposizione altri importanti servizi.

1. **Host aliasing**. Un host dal nome complicato può avere uno o più sinonimi, detti *alias*. Per esempio, *relay1.west-coast.enterprise.com* potrebbe avere, diciamo, due sinonimi quali *enterprise.com* e *www.enterprise.com*. In questo caso, si dice che il nome *relay1.west-coast.enterprise.com* è un **hostname canonico**. Gli alias sono generalmente più facili da ricordare rispetto agli hostname canonici. Il DNS può essere invocato da un'applicazione per ottenere l'hostname canonico di un sinonimo.
2. **Mail server aliasing**. È auspicabile che gli indirizzi di posta elettronica siano facili da ricordare. Ad esempio se un'utente ha un account Hotmail, il suo indirizzo di posta elettronica potrebbe essere «bob@yahoo.com». Tuttavia, l'hostname del server di posta Hotmail è molto più complicato da ricordare rispetto a *yahoo.com*. Ad esempio, il nome canonico potrebbe essere simile a *relay1.west-coast.yahoo.com*. Un'applicazione di posta può invocare il DNS per ottenere il nome canonico di un sinonimo fornito.
3. **Distribuzione del carico di rete (load distribution)**. Il DNS viene utilizzato per distribuire il carico tra server replicati, per esempio dei web server. I siti con molto traffico, vengono replicati su più server, ognuno eseguito su un host diverso con indirizzo IP differente. Nel caso di web server replicati, va associato a ogni hostname canonico un insieme di indirizzi IP. Il database DNS contiene

questo insieme di indirizzi. Quando i client effettuano una query DNS per un nome associato a un insieme di indirizzi, il server risponde con l'intero insieme di indirizzi, ma ne varia l'ordinamento a ogni risposta. Dato che un client invia il suo messaggio di richiesta HTTP al primo indirizzo IP elencato nell'insieme, la rotazione DNS distribuisce il traffico sui server replicati. La rotazione viene anche utilizzate affinchè più server di posta possano condividere lo stesso nome. Inoltre, le società di distribuzione dei contenuti hanno fatto uso del DNS in mdoi più sofisticati per effettuare la diffusione di contenuti web.

DNS è specificato nelle RFC 1034 e 1035 ed è stato aggiornato nelle successive RFC aggiuntive.

3.2.2. Panoramica del funzionamento di DNS

Analizziamo il servizio di traduzione da hostname a indirizzo IP.
Supponiamo che una certa applicazione in esecuzione sull'host di un utente abbia necessità di trudurre un hostname in indirizzo IP. L'applicazione invocherà il lato client del DNS, specificando l'hostname da tradurre. Su molte macchine basate su UNIX, `gethostbyname()` è la chiamata di funzione effettuata da un'applicazione per ottenere il servizio di traduzione. Il DNS sull'host prende poi il controllo, inviando un messaggio di richiesta sulla rete. Tutte le query DNS e i messaggi di risposta vengono inviati all'interno di datagrammi UDP diretti alla porta 53. Dopo un ritardo che varia dai millisecondi ai secondi, il client DNS sull'host dell'utente riceve un messaggio di risposta contenente la corrispondenza desiderata, che viene poi passata all'applicazione che ne ha fatto richiesta. Dal punto di vista dell'applicazione, il DNS è una scatola nera che fornisce un servizio di traduzione semplice e diretto. La scatola nera è costituita da un gran numero di DNS server distribuiti per il mondo e da un protocollo a livello di applicazione che specifica la comunicazione tra DNS server e host richiedenti.
Un primo approccio potrebbe essere quello di utilizzare un DNS server contenente tutte le corrispondenze. Questa semplicità progettuale è inappropriata per l'attuale Internet, dotata di un vasto e sempre crescente numero di host.
Tra i problemi legati a uno schema centralizzato ricordiamo i seguenti.

- **Un solo point of failure** (*punto di vulnerabilità*). Se il DNS server si guasta, ne soffre l'intera Internet.
- **Volume di traffico**. Un singolo DNS server dovrebbe gestire tutte le richieste.
- **Database centralizzato distante**. Un singolo DNS server non può essere vicino a tutti i client.
- **Manutenzione**. Il singolo DNS server dovrebbe contenere record relativi a tutti gli host di Internet. Non solo tale database centralizzato sarebbe vasto, ma dovrebbe essere aggiornato frequentemente per tener conto di ogni nuovo host.

Un database centralizzato su un singolo DNS server non è in grado di adattarsi alla crescita esponenziale della rete. Quindi, il DNS è stato progettato in maniera distribuita.

Un database distrubuito e gerarchico

Per trattare il problema della scalabilità, il DNS utilizza un grande numero di server, organizzati in maniera gerarchica e distribuiti nel mondo. In prima approssimazione, esistono tre classi di DNS server: i **root server**, i **top-level domain (TLD) serve** e i **server autoritativi**, organizzati in una gerarchia, come mostrato nella seguente figura.

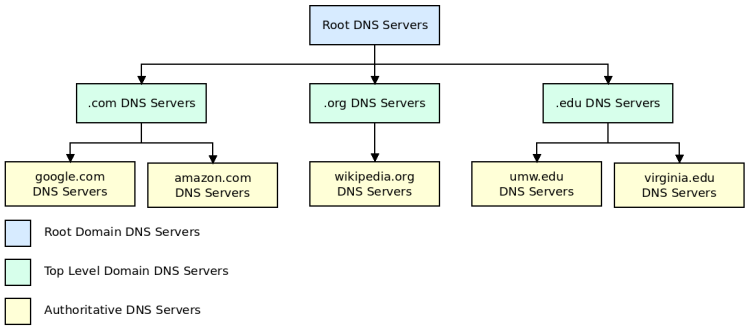


Figura 5: Gerarchia parziale dei server DNS

Per comprendere l'interazione tra le classi, supponiamo che un client DNS voglia determinare l'indirizzo IP relativo all'hostname `www.amazon.com`. Per fare ciò, il client contatta uno dei root server, che gli restituisce uno o più indirizzi IP relativi al TDL server per il dominio `com`. Contatta uno di questi TLD server,che gli restituisce uno o più indirizzi IP del server autoritativo per `amazon.com`. Infine, contatta uno dei server autoritativi per `amazon.com`, che gli restituisce l'indirizzo IP dell'hostname `www.amazon.com`. Analizziamo le tre classi di DNS server.

- **Root server**. In Internet esistono più di 1000 root server, dislocati in tutto il mondo. Tali root server sono copie di 13 differenti root server gestiti da 12 diverse organizzazioni, coordinate attraverso la *Internet Assigned Numbers Authority*. I root server forniscono gli indirizzi IP dei TLD server.
- **Top-level domain (TLD) server**. Questi server si occupano dei domini di primo livello quali com, org, net, edu e gov, e di tutti i domini di primo livello relativi ai vari paesi, come it, uk, fr, ca e jp. I TLD server forniscono gli indirizzi IP dei server autoritativi. L'assegnazione dei domini che indicano una nazione o un tipo di organizzazione è gestita dall'ICANN (Internet Corporation for Assigned Names and Numbers).
- **DNS server autoritativi**. Ogni organizzazione dotata di host pubblicamente accessibili tramite Internet deve fornire record DNS pubblicamente accessibili che associno i nomi di tali host a indirizzi IP. Il DNS server autoritativo dell'organizzazione ospita questi record. Un'organizzazione può scegliere di im-

plementare il proprio server autoritativo o di pagare un fornitore di serverizi per ospitare questi record su un suo server. La maggior parte delle università e delle grandi società implementa e gestisce dei propri server autoritativi primario e secondario.

I DNS server appena descritti sono tutti collocati in una gerarchia di DNS server. Esiste un altro importante tipo di DNS, detto **DNS server locale**, che non appartiene strettamente alla gerarchia di server, ma che è comunque centrale nell'architettura DNS. Ciascun ISP ha un DNS server locale, detto anche **default name server**. Quando un host si connette a un ISP, quest'ultima gli fornisce un indirizzo IP tratto da uno o piu dei suoi DNS server locali, generalmente tramite DHCP. Si può facilmente determinare l'indirizzo IP del proprio DNS server locale accedendo alla finestra di stato della rete in Windows o UNIX. Un DNS server locale è solitamente «vicino» all'host. Quando un host effettua una richiesta DNS, la query viene inviata al DNS server locale, che opera da proxy e inoltra la query alla gerarchia dei DNS server.
Le richieste DNS possono essere di due tipi:

- **query ricorsive**: è una richiesta in cui un client chiede a un server DNS (detto *resolver ricorsivo*) di ottenere l'indirizzo IP completo e definitivo per un nome di dominio, oppure di restituire un errore se non lo trova. Il server ha tutta la responsabilità: se non ha il dato in cache, contatta altri server per conto del client finchè non ottiene la risposta definitiva e gliela consegna.
- **query iterativa**: il server DNS interpellato non fa tutto il lavoro per conto del richiedente. Se conosce la risposta la fornisce, altrimenti, resituisce la «migliore risposta possibile», ovvero un **rinvio (referral)**, indicando al client quale altro server DNS conviene contattare subito dopo. Il client stesso prosegue l'esplorazione passo dopo passo.

I root server e i TLD server sono generalmente configurati per essere non-ricorsivi e non accettano query ricorsive a causa dell'eccessivo traffico.

DNS Caching

Il **DNS caching** è una caratteristica di fondamentale importanza. Il DNS sfrutta in modo estensivo il caching per migliorare le prestazioni di ritardo e per ridurre il numero di messaggi DNS che «rimbalzano» su Internet. L'idea del DNS caching è semplice. In una concatenazione di richieste, il DNS server che riceve una risposta DNS può mettere in cache le informazioni contenute.
Se una ⟨hostname/indirizzo IP⟩ è nella cache di un DNS server e giunge al server un'altra richiesta con lo stesso hostname, il DNS server può fornire l'indirizzo IP desiderato, anche se non è autoritativo per tale indirizzo. Dato che gli host e le associazioni tra nome e indirizzo IP non sono in alcun modo permanenti, i DNS server invalidano le informazioni in cache dopo un periodo di tempo fissato. Un DNS

server locale, può, inoltre, memorizzare in cache gli indirizzi IP dei TLD server, consentendogli di aggirare i root server nella catena di richieste.

3.2.3. Record e messaggi DNS

I server che implementano il database distribuito di DNS memorizzano i cosiddetti **record di risorsa** (RR, *resource record*), tra cui quelli che forniscono le corrispondenze tra nomi e indirizzi. Ogni messaggio di risposta DNS trasporta uno o più record di risorse. Un record di risorsa contiene i seguenti campi:

(Name, Value, Type, TTL)

TTL è il time to live, ossia il tempo residuo di un record e determina quando una risorsa vada rimossa dalla cache. Il significato di `Name` e `Value` dipene da `Type`:

- Se `Type=A`, allora `Name` è il nome dell'host e `Value` è il suo indirizzo IP. Pertanto un record di tipo A fornisce la corrispondenza tra hostname standard e indirizzo IPv4.
- Se `Type=AAAA`, allora `Name` è il nome dell'host e `Value` è il suo indirizzo IPv6.
- Se `Type=NS`, allora `Name` è un dominio e `Value` è l'hostname del DNS server autoritativo che sa come ottenere gli indirizzi IP degli host nel dominio. Questo record viene usato per instradare le richieste DNS successive alla prima nella concatenazione delle query.
- Se `Type=CNAME`, allora `Value` rappresenta il nome canonico dell'host per il sinonimo `Name`. Questo record può fornire agli host richiedenti il nome canonico relativo a un hostname.
- Se `Type=MX`, allora `Value` è il nome canonico di un mail server che ha il sinonimo `Name`. Questo tipo di record consente agli hostname dei mail server di avere sinonimi semplici. Per ottenere il nome canonico del server di posta, un client DNS dovrebbe interrogare un record MX. Per ottenere il nome canonico dell'altro server, il client DNS dovrebbe interrogare il record CNAME.

Un DNS server autoritativo per un certo hostname contiene un record di tipo A per l'hostname. Anche se il server non fosse autoritativo, potrebbe tuttavia contenere un record di tipo A nella propria cache. Se un server non è quello autoritativo per un certo hostname, allora conterrà un record di tipo NS per il dominio che include l'hostname, e conterrà anche un record di tipo A che fornisce l'indirizzo IP del DNS server nel campo `Value` del record NS.

Messaggi DNS

Abbiamo fatto riferimento agli unici due tipi di messaggio DNS: le query e i messaggi di risposta che presentano, entrambi, lo stesso formato, mostrati nella seguente figura.

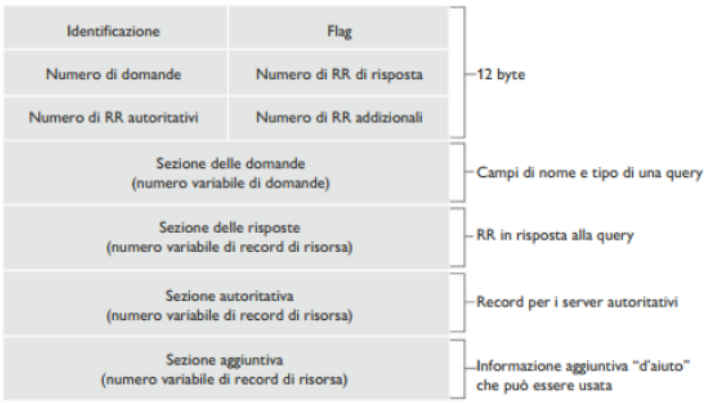


Figura 6: Formato dei messaggi DNS

La semantica dei campi dei messaggi DNS è la seguente.

- I primi 12 byte rappresentano la *sezione di intestazione (header section)*, che a sua volta contiene un certo numero di campi.
  - Il primo è un numero di 16 bit che identifica la richiesta. Tale identificatore viene copiato nei messaggio di risposta a una richiesta, consentendo al client di far corrispondere le risposte ricevute con le query inviate.
  - Il campo flag è composto da una serie di bit: il primo, il bit di richiesta/risposta indica se è il messaggio è una richiesta (0) o una risposta (1). Un bit viene impostato nei messaggi di risposta quando il DNS server è autoritativo per il nome richiesto. Un ulteriore bit, chiamato **richiesta di ricorsione**, viene impostato quando un client desidera che il DNS server effettui ricorsione quando non dispone del record. Si imposta un campo da 1 bit di ricorsione disponibile all'interno di una risposta se il DNS server supporta la ricorsione.

  Nell'intestazione troviamo anche quattro campi il cui nome comincia con «numero di», che indicano il numero di occorrenze delle quattro sezioni di tipo dati che seguono l'intestazione.
- La *sezione delle domande* contiene informazioni sulle richieste che stanno per essere effettuate. Inoltre, include
  - un campo nome con il nome che sta per essere richiesto;
  - un campo tipo che indica il tipo della domanda sul nome.
- In una risposta proveniente da DNS server, la *sezione delle risposte* contiene i record di risorsa relativi al nome originariamente richiesto. Una risposta può restituire più RR, dato che un hostname può avere più indirizzi IP. È il caso dei web server replicati.
- La *sezione autoritativa* contiene i record di altri server autoritativi.
- La *sezione aggiuntiva* racchiude altri record utili: ad esempio, se il campo di risposta relativo a una richiesta MX contiene un record di risorsa che fornisce

l'hostname canonico del server di posta, la sezione aggiuntiva contiene un record di tipo A che fornisce l'indirizzo IP relativo all'hostname canonico del server di posta.

3.2.4. Registrazione di un dominio e Zone Transfer

Per inserire un nuovo dominio nel database DNS (ad esempio `networkutopia.com`), è necessario rivolgersi a un **registrar**. Un registrar è un'azienda commerciale, tipicamente accreditata dall'ICANN (Internet Corporation for Assigned Names and Numbers), che verifica l'unicità del nome di dominio, richiede un compenso per il servizio e si occupa di inserirlo nel sistema DNS.
Al momento della registrazione, l'utente deve fornire al registrar i nomi e gli indirizzi IP dei propri DNS server autoritativi, generalmente un primario e un secondario (es. `dns1.networkutopia.com` e `dns2.networkutopia.com`, insieme ai relativi IP). Il registrar provvede a inserire nei TLD server di competenza (ad esempio i server responsabili per il suffisso `.com`) due tipologie di record: i record di tipo NS e quelli di tipo A. Nello specifico, per il server primario dell'esempio verrebbero inseriti i seguenti record di risorsa:

- `(networkutopia.com, dns1.networkutopia.com, NS)`
- `(dns1.networkutopia.com, 212.2.212.1, A)`

Successivamente, è responsabilità del proprietario del dominio assicurarsi che nei propri server autoritativi vengano inseriti i record di risorsa specifici per i servizi ospitati. Ad esempio, deve essere configurato il record di tipo A per il server web (`www.networkutopia.com`) e il record MX per il server di posta elettronica (`mail.networkutopia.com`). L'aggiunta di questi record può avvenire configurando staticamente i server oppure, più recentemente, in modo dinamico inviando messaggi DNS grazie all'opzione UPDATE (specificata nelle RFC 2136 e 3007).
Completata la configurazione, la risoluzione procede normalmente: il DNS server locale di un utente che vuole visitare il sito interrogherà il TLD server, il quale risponderà fornendo l'IP del server autoritativo grazie ai record precedentemente inseriti dal registrar. Interrogando infine il server autoritativo, il client otterrà l'indirizzo IP del web server desiderato.

Zone Transfer

Per garantire disponibilità e affidabilità, i record DNS appena creati devono essere replicati. I record vengono copiati dal server autoritativo primario a quelli secondari tramite un'operazione chiamata **Zone Transfer**. Essendo un'operazione che comporta spesso il trasferimento di un'intera zona e di molti dati (superando il classico limite di 512 byte del protocollo UDP), il trasferimento avviene in modo affidabile su protocollo TCP. Le query utilizzate dai server secondari per allinearsi al primario sono principalmente due:

- **AXFR**: per richiedere una copia completa e integrale dei record della zona.

- **IXFR**: per richiedere un trasferimento incrementale, ottenendo solamente le modifiche più recenti.

Livello di Trasporto

4.1. Introduzione e servizi a lviello di trasporto

Un protocollo a livello di trasporto mette a disposizione una **comunicazione logica** tra processi applicativi di host differenti.
Per comunicazione logica si intende, dal punto di vista dell'applicazione, che tutto proceda come se gli host che eseguono i processi fossero direttamente conessi. I processi applicativi usano la comunicazione logica fornita dal livello di trasporto per scambiare messaggi, senza preocupparsi dei dettagli dell'infrastruttura fisica utilizzata per trasposrtarli.
I protocolli a livello di trasporto sono implementati nei sistemi periferici, ma non nei router della rete. Nel mittente, il livello di trasporto converte i messaggi che riceve da un processo applicativo in pacchetti a livello di trasposrto, noti come **segmenti**. Questo avviene spezzando i messaggi applicativi in parti più piccole e aggiungendo a ciascuna di esse un'intestazione di trasporto per creare un segmento. Il livello di rete del ricevente estrae il segmento del datagramma e lo passa al livello di trasporto. Quest'ultimo elabora il segmento ricevuto, rendendo disponibili all'applicazione destinataria i dati del segmento.
Internet fornisce due protocolli a livello di trasporto, TCP e UDP, che forniscono servizi diversi alle applicazioni che ne fanno uso.

4.1.1. Relazione tra i livello di trasporto e di rete

Il livello di trasposrto è collocato sopra quello di rete nella pila di protocolli, ricordiamo la **Figura 3**. Un livello di trasporto mette a disposizione una comunicazione logica tra processi che vengono eseguiti su host diversi, un protocollo a livello di rete fornisce comunicazione logica tra host. La distinzione è sottile, ma importante. Determinati servizi possono essere garantiti da un protocollo di trasporto anche se il sottostante protocollo di rete non offre un servizio corrispondente. Ad esempio, un protocollo di trasporto può offrirre il servizio di trasferimento dati affidabile anche quando il sottostante protocollo di rete non lo è. Considerando un altro aspetto, un protocollo di trasposrto può usare la cifratura per garantire che i messaggi delle applicazioni non vengano letti da intrusi, anche quando il livello di rete non può garantire la riservatezza dei segmenti a livello di trasporto.

4.1.2. Panoramica del livello di trasporto di Internet

Internet mette a disposizione del livello di applicazione due protocolli. Uno è **UDP**, *User Datagram Protocol*, che fornisce alle applicazioni un servizio non affidabile e non orientate alla connessione, l'altro è **TCP**, *Transmission Control Protocol*, che offre un servizio affidabile e orientato alla connessione.
Prima di entrare nel dettaglio dei due protocolli è meglio analizzare brevemente il livello di rete. Il protocollo di rete in Internet ha un nome: IP (Internet Protocol). Tale protocollo fornisce comunicazione logica tra host. Il suo modello di servizio prende il nome di **best-effort delivery service** o, **best-effort**. Questo significa che IP fa del suo meglio per consegnare i segmenti tra host comunicanti ma non offre garanzie. NOn assicura né la consegna dei segmenti né il rispetto dell'ordine originario e non garantisce neppure l'integrità dei dati all'interno dei segmenti. Per queste ragioni si dice che IP offra un **servizio non affidabile**.
Il compito dei protocolli TCP e UDP è l'estensione del servizio di consegna di IP «tra sistemi periferici» a quello di consegna «tra processi in esecuzione sui sistemi periferici». Il passaggio, da consegna host-to-host a consegna process-to-process, viene detto **multiplexing** e **demultiplexing a livello di trasporto**. Inoltre, TCP e UDP, forniscono un controllo di integrità includendo campi per il riconoscimento di errori nelle intestazioni dei propri segmenti. UDP, come IP, costituisce un servizio inaffidabile: non garantisce che i dati inviati da un processo arrivino intatti (e neanche che arrivino) al processo destinatario.
TCP offre diversi servizi aggiuntivi. Innanzitutto, fornisce un **trasferimento dati affidabile** e, quindi, garantisce che i dati vengano trasferiti da un processo a un altro in modo corretto e ordinato. Pertanto TCP converte il servizio inaffidabile tra sistemi periferici, tipico di IP, in un servizio affidabile di trasporto dati tra processi. TCP fornisce anche il **controllo di congestione**, un servizio offerto alle applicazioni e a Internet nel suo complesso. In pratica, evita che le connessioni TCP intasino i collegamenti e i rotuer tra gli host comunicanti con un'eccessiva quantità di traffico. Questo è reso possibile regolando la velocità alla quale il lato mittente della connessione TCP immette traffico in rete. Il traffico UDP, al contrario, non viene regolato. Le applicazioni che usano UDP possono spedire a qualsiasi velocià desiderata e per tutto il tempo voluto.

4.2. Multiplexing e demultiplexing

Analizziamo il servizio di multiplexing e demultiplexing in Internet, anche se è un servizio presente in tutte le reti di calcolatori.
Nell'host destinatario il livello di trasposrto riceve segmenti dal livello di rete immediatamente sottostante. Il livello di trasposrto ha il compito di consegnare i dati di questi segmenti al processo applicativo appropriato in esecuzione nell'host.

Esaminiamo come viene realizzato.
Ricordiamo che un processo può gestire una o più **socket**, attraverso le quali i dati fluiscono dalla rete al processo e viceversa. Di conseguenza, il livello di trasposrto nell'host di ricezione non trasferisce i dati direttamente a un processo, ma a una socket che fa da intermediario. Siccome, a ogni dato istante, può esserci più di una socket nell'host di ricezione, ciascuna avrà un identifcatore univoco il cui formato dipende dal fatto che si tratti di socket UDP o TCP.
Vediamo come l'host in ricezione indirizzi verso la socket appropriata il segmento a livello di trasporto in arrivo. Ciascun segmento a lviello di trasporto ha vari campi che permettono di fare ciò. Nel lato del ricevente, il livello di trasporto esamina questi campi per identificare la socket di ricezione e quindi vi dirige il segmento. Il compito di trasportare i dati dei segmenti a livello di trasporto verso la giusta socket viene detto **demultiplexing**. Il compito di radunare frammenti di dati da diverse socket sull'host di origine e incapsulare ognuno con intestazioni a livello di trasporto per creare dei segmenti e passarli al livello di rete, viene detto **multiplexing**.
Le funzioni di multiplexing e demultiplexing hanno specifico interesse ogni volta che un protocollo a un qualsiasi livello è utilizzato da più entità al livello immediatamente superiore.
Vediamo come le funzioni di multiplexing e demultiplexing vengono realizzati negli host. Da quanto visto in precedenza, il multiplexing a livello di trasporto richiede

1. che le socket abbiano identificatori unici
2. ogni segmenti presenti campi che indichino la socket cui va consegnato il segmento.

Questi sono, come indicati nella figura seguente, il **campo del numero di porta** e il **campo del numero di porta di destinazione**.

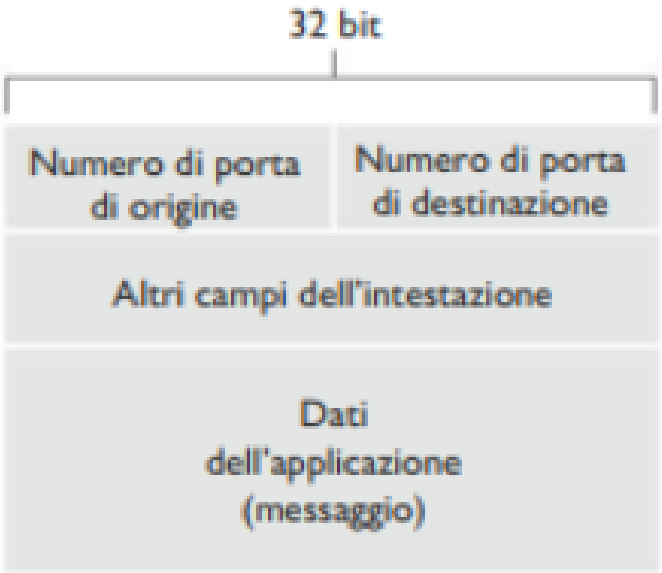


Figura 7: I campi del numero di porta di origine e di destinazione nei segmenti a livello di trasporto

I segmenti UDP e TCP presentano anche altri campi. I numeri di porta sono di 16 bit e vanno da 0 a 65535, quelli che vanno da 0 a 1023 sono chiamati **numeri di porta noti** e sono riservati per essere usati da protocolli applicativi ben noti, come HTTP (porta 80), FTP (porta 21) e così via. L'elenco dei numeri di porta noti è fornito nell'RFC 1700. Le porte da 1024 a 49151 sono **porte registrate**, *registered ports*, che sono registrate da IANA su richiesta di terze parti. Queste sono dette anche *porte utente*, perchè utilizzabili da applicazioni non di sistema. Le porte da 49152 a 65535 sono dette **porte private** o **dinamiche** che vengono assegnate automaticamente e temporaneamente quando un'app client avvia una connessione.

4.2.1. Multiplexing e demultiplexing non orientati alla connessione

Descriviamo il processo di multiplexing/demultiplexing sul protocollo UDP. Supponiamo che un processo nell'host A, con porta UDP 19157, voglia inviare un blocco di dati applicativi a un processo con porta UDP 46428 nell'host B. Il livello di trasporto di A crea un segmento che include i dati applicativi, i numeri di porta di origine e di destinazione e altri valori. Il livello di trasporto passa, il segmenti risultante al livello di rete, che lo incapsula in un datagramma IP, ed effettua un tentivo best-effort di consegna del segmento all'host in ricezione. Se il segmento arriva all'host B, il suo livello di trasporto esamina il numero di porta di destinazione del segmento e lo coonsegna alla propria socket identificata da tale porta. L'host B potrebbe avere in esecuzione più processi, ciascuno con la propria socket UDP e relativo numero di porta. Quando i segmenti UDP giungono dalla rete, l'host B

dirigie ciascun segmento alla socket appropriata.
Una socket UDP viene identificata da una coppia formata da indirizzo IP e numero di porta di destinazione. Due segmenti UDP che hanno diversi indirizzi IP e/o diversi numeri di porta, ma hanno stesso indirizzo IP e stesso numero di porta di destinazione, vengono diretti allo stesso protocollo di destinazione tramite la medesima socket.
Il numero di porta di origine serve come parte di un «indirizzo di ritorno»: quando B vuole restituire il segmento ad A, la porta di destinazione del segmento da B verso A assumerà il valore della porta di origine del segmento da A verso B. L'indirizzo di ritorno completo è costituito dall'indirizzo IP di A più il numero di porta di origine.

4.2.2. Multiplexing e demultiplexing orientati alla connessione

Per comprendere il demultiplexing TCP, dobbiamo analizzare le socket TCP e come si stabiliscono connessioni TCP. Una differenza tra socket TCP e UDP risiede nel fatto che la prima è identificata da quattro parametri: indirizzo IP di origine, numero di porta di origine, indirizzo IP di destinazione e numero di porta di destinazione. Pertanto, quando un segmento TCP giunge dalla rete in un host, quest'ultimo utilizza i quattro valori per dirigere il segmento verso la socket appropriata. Al contrario di UDP, due segmenti TCP in arrivo, aventi indirizzi IP di origine o numero di porta di origine diversi, vengono diretti a due socket differenti, anche a fronte di indirizzo IP e porta di destinazione uguali, con l'eccezione dei segmenti TCP che trasportano la richiesta per stabilire la connessione.
L'host server può ospitare più socket TCP contemporanee collegate a processi diversi, ognuna identificata da una specifica quaterna di valori. Quando il segmento TCP arriva all'host, i quattro campi citati prima vengono utilizzati per fare demultiplexing e dirigere il segmento verso la socket appropriata.
La situazione è schematizzata nella seguente figura:

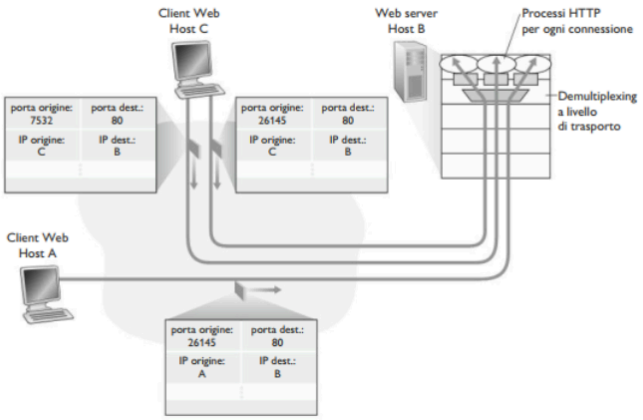


Figura 8: Due client usano stesso numero di porta di destinazione per comunicare con stessa applicazione sul web server

4.3. Trasporto non orientato alla connessione: UDP

Il protocollo UDP fa il minimo che un protocollo di trasporto debba fare. A parte la funzionalità di multiplexing/demultiplexing e una forma di controllo degli errori molto semplice, non aggiunge nulla a IP. Quando lo sviluppatore sceglie UDP anzichè TCP, l'applicazione dialoga quasi in modo diretto con IP. UDP prende i messaggi dal processo applicativo, aggiunge il numero di porta di origine e di destinazione per il multiplexing/demultiplexing, aggiunge altri due piccoli campi e passa il segmento risultante a livello di rete. Questi incapsula il segmento in un datagramma IP e quindi effettua un tentativo di consegnarlo all'host di destinazione in modalità best-effort. Se il segmento arriva a destinazione, UDP utilizza il numero di porta di destinaizone per consegnare i dati del segmento al processo applicativo corretto. Notiamo che in UDP non esiste handhsaking tra le entità di invio di ricezione a lviello di trasporto. Infatti si dice che UDP è *non orientato alla connessione*. UDP fornisce un sercizio senza connessione non solo unicast, ma anche **multicast** e **broadcast**.
DNS è un tipico esempio di protocollo applicativo che utilizza UDP. Anche **SNMP (Simple Network Management Protocol)** si affida ad UDP e anche applicazioni basate su semplici servizi richiesta-risposta.
Ci sono diversi motivi per i quali uno svilupattore dovrebbe scegliere di costruire un'aplicazione su UDP anzichè su TCP.

- *Controllo più preciso a livello di applicazione su quali dati sono inviati e quando.* Non appena un processo applicativo passa dei dati a UDP, quest'ultimo li impacchetta in un segmento che trasferisce immediatamente al livello di rete. TCP, invece, dispone di un meccanismo di controllo della congestione che ritarda l'invio a livello di trasposrto quando uno o più collegamenti tra origine e destinazione diventano eccessivamente congestionati. TCP continua, inoltre, a inviare il segmenti fino a quando viene notificata la sua ricezione da parte della destinazione. Dato che le applicazioni in tempo reale spesso richiedono una velocità minima di invio e non sopportano ritardi eccessivi e tollerano una certa perdita di dati, il modello TCP non si adatta bene a queste esigenze.
- *Nessuna connessione stabilita.* TCP utilizza un handshake a tre vie prima di iniziare il trasferimento dei dati. UDP invece invia dati a raffica senza alcun preliminare formale. Pertanto, UDP non introduce alcun ritardo nello stabilire una connessione. Questo è, probabilemente, il motivo per cui DNS utilizza UDP anzichè TCP. HTTP, invece, usa TCP, dato che l'affidabilità risulta critica per le pagine web con testo. Il protocollo QUIC (Quick UDP Internet Protocol) utilizza UDP come protocollo di trasporto e implementa l'affidabilità in un protocollo a livello di applicazione.

- *Nessun stato di connessione.* TCP mantiene lo stato della connessione nei sistemi perferici. Questo stato include buffer di ricezione di invio, parametri per il controllo di congestione e parametri sul numero di sequenza e di acknowledgement. UDP, invece, n on conserva lo stato della connessione e non tiene traccia di questi parametri. Per questo motivo, un server dedicato a una particolare applicazione può generalmente supportare molti più client attivi quando l'applicazione utilizza UDP anzichè TCP.
- *Minor spazio usato per l'intestazione del pacchetto.* L'intestazione dei pacchetti TCP aggiunge 20 byte, mentre UDP solo 8.

La seguente figura elenca alcune diffuse applicazioni per Internet e i relativi protocolli di trasporto usati.

Applicazione	Protocollo a livello di applicazione	Protocollo di trasporto sottostante
Posta elettronica	SMTP	TCP
Accesso a terminali remoti	Telnet	TCP
Accesso sicuro a terminali remoti	SSH	TCP
Web	HTTP, HTTP/3	TCP (per HTTP), UDP (per HTTP/3)
Trasferimento file	FTP	TCP
Server di file remoti	NFS	generalmente UDP
Dati multimediali in streaming	DASH	TCP
Telefonia su Internet	generalmente proprietario	UDP o TCP
Gestione di rete	SNMP	generalmente UDP
Traduzione di nomi	DNS	generalmente UDP

Figura 9: Protocolli di trasporto delle più diffuse applicazioni Internet

Prima di analizzare la struttura dei segmenti UDP menzionamio che le applicazioni possono ottenere un trasferimento dati affidabile anche con UDP. Ciò avviene se l'affidabilità è insita nell'applicazione stessa. Il protocollo QUIC utilizza UDP come protocollo di trasporto e implementa l'affidabilità in un protocollo a livello di applicazione. Avere affidabilità direttamente nell'applicazione consentirebbe ai processi applicativi di comunicare in modo affidabile senza essere soggetti ai vincoli sulla velocità di trasmissione imposti dai meccanismi del controllo di congestione di TCP.

4.3.1. Struttura dei segmenti UDP

L'RFC 768 definisce la struttura dei segmenti UDP, come mostrato nella seguente figura:

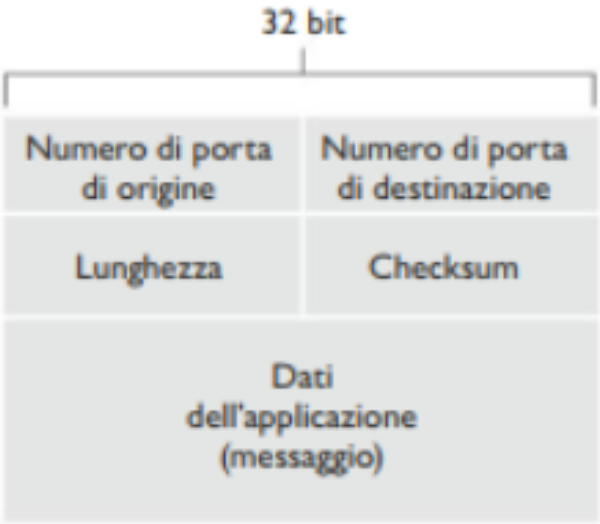


Figura 10: Struttura dei segmenti UDP

L'intestazione UDP presenta solo quattro campi di due byte ciascuno. Il campo *Lunghezza* specifica il numero di byte del segmento UDP e la lunghezza massima è $\leq$ 64 KB. Un valore esplicito di lunghezza è necessario perchè la grandezza del campo *Dati* può essere diversa tra un segmento e quello successivo. L'host ricevente utilizza il checksum per verificare se sono avvenuti errori nel segmento. In realtà, oltre che sul segmento UDP, il checksum è calcolato anche su alcuni campi dell'intestazione IP.

4.3.2. Checksum UDP

Il checksum UDP server per il ritrovamento degli errori. In altre parole, viene utilizzato per determinare se i bit del segmento UDP sono stati alterati durante il loro trasferimento da sorgente a destinazione. Il lato mittente UDP effettua il complemento a 1 della somma di tutte le parole da 16 bit nel segmento, e l'eventuale riporto finale viene sommato al primo bit. Tale risultato viene posto nel campo *Checksum* del segmento UDP.

Esempio 1. Supponiamo di avere le seguenti tre parole di 16 bit

0110011001100000
0101010101010101
1000111100001100

La somma delle prime due è:

$$\begin{array}{r} 0110011001100000 + \\ \underline{0101010101010101 =} \\ 1011101110110101 \end{array}$$

Sommando la terza parola al risultato precedente otteniamo:

$$\begin{array}{r} 1011101110110101 + \\ \underline{1000111100001100 =} \\ 0100101011000010 \end{array}$$

Notiamo che il riporto di quest'ultima è stato sommato al primo bit. Il complemento a 1 si ottiene convertendo i bit 0 in 1 e viceversa. Di conseguenza, il checksum sarà 1011010100111101. In ricezione, si sommano le tre parole iniziali e il checksum. Se non ci sono errori nel pacchetto, l'addizione darà 1111111111111111, altrimenti se un bit vale 0 sappiamo che è stato introdotto almeno un errore nel pacchetto.

Il motivo per il quale UDP metta a disposizione un checksum è che non c'è garanzia che tutti i collegamenti tra origine e destinazione controllino gli errori. Inoltre, anche se i segmenti fossero trasferiti correttametne lungo un collegamento, si potrebber verificare un errore mentre il segmento si trova nella memoria di un router. Dato che non sono garantiti né affidabilità del singolo collegamento né il rilevamento di errori in memoria, UDP deve mettere a disposizione a livello di trasporto un meccanismo di verificare su base end-to-end se si vuole che il servizio trasferimento dati sia ing rado di rilevare eventuali errori. Questo è un esempio del principio **end-to-end** nella progettazione di sistemi in base al quale, dato che determinate funzionalità devono essere implementate su base end-to-end, le funzioalità posizionate ai livelli inferiori possono diventare ridondanti o di scarso valore se confrontate con le stesse funzioanalità offerete dai livelli superiori.
UDP non a nulla per risolvere le situazioni di errore; alcune implementazioni UDP si limitano a scartare il segmento danneggiato, altre lo trasmettono all'applicazione con un avvertimento.
Il controllo degli errori basato su checksum in UDP è **opzionale in IPv4** mentre in **IPv6 è obbligatorio**.

4.4. Trasporto orientato alla connessione: TCP

Per offrire trasferimento dati affidabile, TCP si basa su principi fondamentali come la rilevazione degli errori, le ritrasmissioni, gli acknowledgement cumulativi, i contatori e i campi nell'intestazione per numeri di sequenza e acknowledgement.

4.4.1. Connessione TCP

TCP viene detto **orientato alla connessione** in quanto, prim di effettuare lo scambio dei dati, i processi devono effettuare l'handshake, ossia devono inviarsi reciprocamente alcuni segmenti preliminari per stabilire i parametri del successivo trasferimento dati.
La connessione TCP non è un circuito end-to-end TDM o FDM, come in una rete a commutazione di circuito, in quanto lo stato della connessione riside completamente nei due sistemi perfierici e non negli elementi di rete intermedi, questi ultimi non salvano lo stato della connessione TCP.
Una connessione TCP offre un **servizio full-duplex**: su una connessione TCP tra il processo A su un host e il processo B su un altro host, i dati a livello di applicazione possono fluire dal processo A al processo B nello stesso momento in cui fluiscono in direzione opposta. Una connessione TCP è anche **punto a punto**, ossia ha luogo tra singolo mittente e singolo destinatario.
Per instaurare una connessione TCP, il client invia per primo uno speciale segmento TCP; il server rispende con un secondo segmento speciale TCP; infine il client risponde con un terzo segmento speciale. I primi due non trasportano payload, ossia non hanno dati a livello applicativo; il terzo può invece trasportare informzioni utili. Dato che i due host si scambiano tre segmenti, questa procedura viene chiamata **handshake a tre vie**.
Una volta instaurata una connessione TCP, i due processi applicativi si possono scambiare dati. Consideriamo l'invio di dati dal processo client al processo server. Il primo manda un flusso di dati attraverso la socket. Questi, quando hano attraversato il punto di uscita, sono nelle mani di TCP in esecuzione nel client. TCP dirige i dati al **buffer di invio** della connessione, uno dei buffer riservato durante l'handshake a tre vie, da cui, preleverà blocchi di dati e li passerà al livello di rete.
Le specifiche TCP non indicano quando TCP debba inviare i dati nel buffer, ma afferma che TCP dovrebbe spedire tali dati in segmenti quando è più conveniente. La massima quantità di dati prelevabili e posizionabili in un segmento viene limitata dalla **dimensione massima di segmento (MSS)**. Questo valore viene impostato determinando prima la lunghezza del grame più grande che può essere inviato a livello di collegamento dall'host mittente locale, la cossidetta **unità trasmissiva massima (MTU)** e poi scegliendo un MSS tale che il segmento TCP stia all'interno di un singolo frame a livello di collegamento, considerando anche la lunghezza dell'intestazione TCP/IP normalmente pari a 40 byte.
Con IPv6 la formula tipica per calcolare l'MSS è

$$\text{MSS} = \text{MTU} - 60$$

TCP accoppia ogni blocco di dati del client a un'intestazione TCP, andando a formare **segmenti TCP**. Questi vengono passati al sottostante livello di rete, dove sono incapsulati nei datagrammi IP a livello di rete che vengono poi immessi

nella rete. Quando all'altro capo TCP riceve un segmento, i dati dael segmento vengono memorizzati nel buffer di ricezione della connessione TCP. Ogni lato della connessione presenta un proprio buffer di invio e di ricezione.
TCP offre un'**interfaccia a stream** (sequenza continua di byte) e non c'è alcuna garanzia che i dati arrivino al destinatario suddivisi negli stessi blocchi con cui sono stati spediti.

4.4.2. Struttura dei segmenti TCP

Esaminiamo la struttura dei segmenti TCP. Il segmento TCP consiste di campi *Intestazione* e di un campo contenente un blocco di dati proveniente dall'applicazione. Come detto prima, la MSS limita la dimensione massima del campo *Dati* di un segmento. TCP, quando invia un file di grandi dimensioni, di solito frammenta il file in porizioni di dimensione MSS.
La seguente figura mostra la **struttura dei segmenti TCP**.

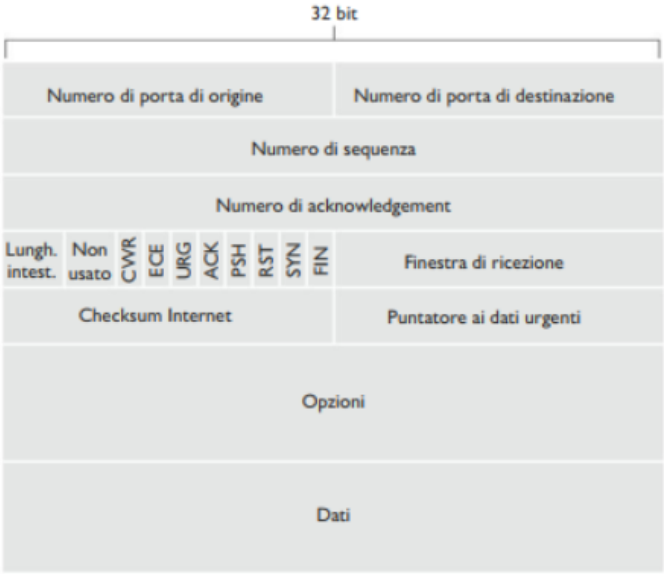


Figura 11: Struttura dei segmenti tcp

Come in UDP, l'intestazione include **numeri di porta origine** e di **destinazione**, utilizzati per il multiplexing/demultiplexing dei dati da e verso le applicazioni del livello superiore, e un campo **Checksum**. L'intestazione dei segmenti TCP comprende poi i seguenti campi.

- Il campo **Numero di sequenza** e il campo **Numero di acknowledgement**, entrambi di 32 bit, vengono utilizzati dal mittente e dal destinatario TCP per implementare il trasferimento dati affidabile.
- Il campo **Finestra di ricezione**, di 16 bit, viene utilizzato per il controllo di flusso.

- Il campo **Lunghezza dell'intestazione**, di 4 bit, specifica la lunghezza dell'intestazione TCP in multipli di 32 bit. L'intestazione TCP ha lunghezza variabile a causa del campo delle opzioni TCP. Generalmente il campo opzioni è vuoto e quindi la lunghezza consueta è di 20 byte. La lunghezza può variare da 20 a 60 byte, poichè il campo *Options* ha una lunghezza variabile di massimo 40 byte.
- Il campo **Opzioni**, facoltativo e di lunghezza variabile, viene utilizzato quando mittente e destinatario negoziono la dimensione massima del segmento (MSS) o come fattore di scala per la finestra nelle reti ad alta velocità. Inoltre, viene definita l'opzione di time-stamping.
- Il campo **Flag** è di 6 bit. Il bit **ACK** viene usato per indicare che il valore trasportato nel campo di acknowledgement è valido. I bit **RST, SYN** e **FIN** vengono utilizzati per impostare e chiudere la connessione. I bit **CWR** ed **ECE** sono usati nel controllo di congestione esplicito. Se il bit **PSH** ha valore 1 il destinatario dovrebbe inviare immediatamente i dati al livello superiore. Infine, si utilizza il bit **URG** per indicare nel segmento la presenza di dati che l'enetità mittente a livello superiore ha marcato come «urgenti». La posizione dell'ultimo byte di dati urgenti viene denotata dal campo **Puntatore ai dati urgenti**, di 16 bit. Quando ci sono dati urgenti, TCP deve informare l'entità destinataria al livello superiore e passarle un puntatore alla fine dei dati urgenti.

Numeri di sequenza e numeri di acknowledgement

Due tra i campi più importanti dell'intestazione del segmento TCP contengono il numero di sequenza e il numero di acknowledgement, che rappresentano una parte critica del servizio di trasferimento dati affidabile proprio di TCP.

TCP vede i dati come un flusso di byte non strutturati, ma ordinati. L'uso dei numeri di sequenza in TCP riflette questa visione, in quanto i numeri di sequenza si applicano al flusso di byte trasmessi. Il **numero di sequenza per un segmento** è pertanto il numero nel flusso di byte del primo byte del segmento.

Consideriamo ora i numeri di acknowledgement. Ricordiamo che TCP è full-duplex, di conseguenza l'host A può contemporaneamente inviare e ricevere dasti dall'host B. I segmenti che provengono dall'Host B presentano un numero di sequenza relativo ai dati che fluiscono da B ad A. Il *numero di acknowledgement che l'host A attende dall'host B*. Consideriamo alcuni esempi.

Supponiamo che l'host A abbia ricevuto da B tutti i byte numerati da 0 a 535 e che A stia per mandare un segmento all'host B. L'host A è in attesa del byte 536 e dei successivi byte nel flusso di dati di B. Pertnato, l'host A scrive 536 nel campo di acknowledgement del segmento che spedisce a B.

Supponiamo, adesso, che l'host A abbia ricevuto un segmento dall'host B contenente i byte da 0 a 535 e un altro segmento contenente i byte da 900 a 1000 ma, per qualche motivo, non ha ricevuto i byte da 536 a 899. In questo esempio, l'host A sta ancora

attendendo il byte 536 per ricreare il flusso di dati di B. Perciò il prossimo segmento di A destinato a B conterrà 536 nel campo del numero di acknowledgement. Dato che TCP effettua l'acknowledgement solo dei byte fino al primo byte mancante nel flusso, si dice che tale protocollo offre **acknowledgement cumulativi**.
Se un host, ad esempio, riceve prima il terzo segmento e non il secondo, ovvero arrivano in disordine, gli RFC non impongono regole su questo aspetto e lasciano la decisione a chi implementa TCP. Esistono due scelte:

1. il destinatario scarta immediatamente i segmenti non ordinati;
2. il destinatario mantiene i byte non ordinati e attende quelli mancanti per colmare i vuoti.

Chiaramente, la seconda scelta è più efficiente in termini di banda occupata e rappresenta l'approccio solitamente utilizzato.

Telnet

Telnet è un protocollo a livello di applicazione per il login remoto che utilizza TCP. Oggigiorno molti utenti preferiscono usare il protocollo SSH, *secure shell*, anzichè Telnet, in quanto con quest'ultimo i dati inviati non vengono criptati rendendolo vulnerabile agli attacchi di intercettazione.
Una comunicazione Telnet avviene nel seguente modo mostrato in figura.

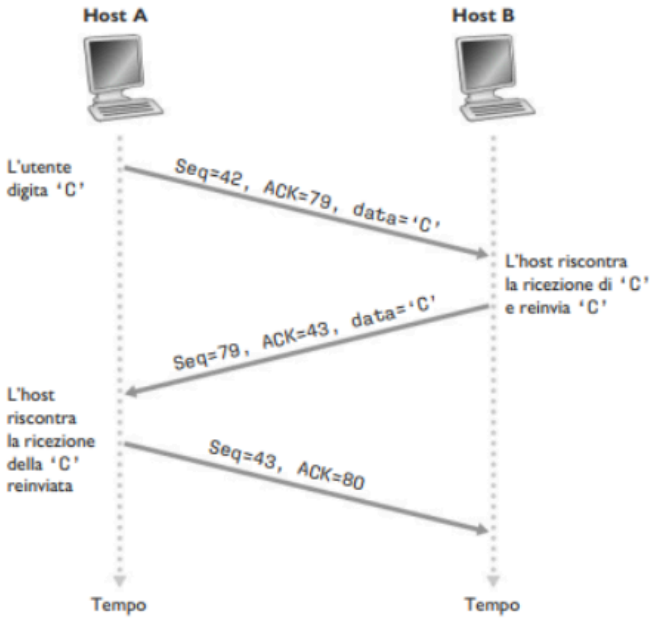


Figura 12: Numeri di sequenza e acknowledgement per applicazioni Telnet su TCP

Notiamo che ogni carattere immesso dall'utente verrà spedito all'host remoto che restituirà una copia di ciascun carattere.

Notiamo che vengono spediti tre segmenti. Il primo è inviato dal client al server e contiente nel campo dati il byte ascii per la lettera "C". Il campo numero di sequenza del primo segmento contiene 42 e, inoltre, visto che il client non ha ancora rivevuto dati dal server, questo primo segmento avrà 79 nel proprio campo di acknowledgement.
Il secondo segmento spedito dal server al client ha un duplice scopo. Innanzitutto, fa un acknowledgement dei dati ricevuti dal server. Ponendo 43 nel camo di acknowledgement, il server indica al client di aver ricevuto con successo i primi 42 byte e di attendere i byte dal 43 in poi. L'altra finalità di questo segmento è di mandare indietro una «eco» della lettera "C". Il secondo segmento ha numero di sequenza 79, ossia il numero iniziale del flusso di dati da server a client in questa connessione TCP. Notiamo che l'acknowledgement dei dati dal client al server viene trasportato in un segmento che a sua volta trasporta dati dal server al client; si dice che tale acknowledgement è **piggybacked** sul segmento dati dal server al client.
Il terzo segmento viene inviato dal client al server e ha come scopo dare un acknowledgement ai dati inviati dal server. Il segmento ha 80 nel suo campo di numero di acknowledgement, in quanto il client ha ricevuto con successo il flusso di byte fino al numero di sequenza 79 e ora è in attesa dei byte dall'80 in avanti.

4.4.3. Timeout e stima del tempo di andata e ritorno (RTT)

TCP utilizza un meccanismo di timeout e ritrasmissione per recuperare i segmenti persi. Sebbene tutto questo sia concettualmente semplice, l'implementazione crea alcuni problemi. La questione più ovvia è la durata degli intervalli di timeout. Il timeout dovrebbe essere più grande del tempo di andata e riutorno sulla connessione (**RTT, Round-Trip Time**), ossia del tempo trascorso da quando si invia un segmento a quando se ne riceve l'acknowledgement, altrimenti ci sarebbero trasmissioni inutili. Il problema è di quanto dovrebbe essere maggiore e come dovrebbe essere stimato RTT.

Stima del tempo di andata e ritorno (RTT)

L'RTT misurato di un segmento, denotato come `SampleRTT`, è la quantità di tempo che intercorre tra l'istante di invio del segmento e quello di ricezione dell'acknowledgement del segmento. ANzichè misurare un `SampleRTT` per ogni segmento, la maggior parte delle implementazioni TCP eeffettua una sola misurazione di `SampleRTT` alla volta. Ossia, in ogni instante di tempo, `SampleRTT` viene valutato per uno solo dei segmenti trasmessi e per cui non si è ancora ricevuto acknowledgement, il che comporta approssimativamente la misurazione di un nuovo valore di `SampleRTT` a ogni RTT. Inoltre, TCP non calcola mai il `SampleRTT` per i segmenti ritrasmessi.
Ovviamente, i campioni variano da segmento a segmento in base alla congestione nei router e al diverso carico sui sistemi periferici. A causa di tale fluttuazione, ogni

valore di SampleRTT può essere atipico.
Per effettuare una stima risulta naturale calcolare una media dei valori si SampleRTT, che TCP chiama EstimatedRTT. Quando si ottiene un nuovo SampleRTT, TCP aggiorna EstimatedRTT secondo la formula:

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

Il valore raccomandato per α è 0.125.
Si noti che EstimatedRTT è una media ponderata dei valori SampleRTT. Tale media attribuisce maggiore importanza ai campioni recenti rispetto a quelli vecchi. In statistica, tale media è detta **media mobile esponenziale ponderata** (EWMA, Exponential Weighted Moving Average).
Oltre ad avere una stima di RTT è importante possedere una misura della sua variabilità. La variazione è denotata con DevRTT e calcolata come:

$$\text{DevRTT} = (1 - \beta) \times \text{DevRTT} + \beta \cdot | \text{SampleRTT} - \text{EstimatedRTT} |$$

Il valore suggerito per β è 0.25s.

Impostazione e gestione del timeout di ritrasmissione
Dati i valori di EstimatedRTT e DevRTT bisogna capire quali valori bisogna utilizzare per l'intervallo di timeout in TCP. L'intervallo non può essere inferiore a EstimatedRTT. Ma l'intervallo stesso non deve essere molto maggiore di EstimatedRTT, altrimenti TCP non ritrasmetterebbe rapidamente il segmento perduto, il che comporterebbe gravi ritardi sul trasferimento dei dati. Quindi bisogna impostare il timeout a EstimatedRTT più un certo margine che dovrebbe essere grande quanto c'è molta fluttuazione nei valori di SampleRTT e piccolo in caso contrario. Qui entra in gioco il valore di DevRTT. L'intervallo di timeout viene calcolato come segue:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

In RFC 6298 viene raccomandato un valore iniziale di TimeoutInterval pari a 1 secondo. Inoltre, quando si verifica un timeout, TimeoutInterval vien raddoppiato per evitare un timeout prematuro riferito a un segmento successivo per cui si riceverà presto un acknowledgement. Quando si riceve un segmento ed EstimatedRTT viene aggiornato, TimeoutInterval viene ricalcolato secondo la formula precedente.

4.4.4. Trasferimento dati affidabile

TCP crea un **servizio di trasporto dati affidabile** al di sopra del servizio inaffidabile e best-effort di IP, assicurando che il flusso di byte che i processi leggono dal buffer di ricezione TCP non sia alterato.
Presentiamo una descrizione molto semplificata di un mittente TCP che fa uso solo di timeout per recuperare i segmenti persi e poi una più completa che utilizza anche gli acknowledgement duplicati. Supponiamo che i dati vengano inviati solo in una

direzione, da A a B, e che il primo stia trasmettendo file di grandi dimensioni.
Esistono tre eventi principali relativi alla trasmissione e ritrasmissione dei dati: dati provenienti dall'applicazione, timeout e ricezione di un ACK. Quando si verifica il primo evento, TCP incapsula i dati che gli giungono dall'applicazione in un segmento e lo passa a IP. Inoltre, se il timer non è gia in funzione per qualche altro segmento, TCP lo avvia quando il segmento viene passato a IP. È utile pensare che il timer sia associato al più vecchio segmento che non ha ricevuto acknowledgement. L'intervallo di scandenza per il timer è il `TimeoutInterval`.
Il secondo evento è il timeout, cui TCP risponde ritrasmettendo il segmento che lo ha causato e quindi riavviato il timer.
Il terzo evento, che deve essere gestito dal mittente TCP, è l'arrivo del segmento di acknowledgement con un valore valido nel campo ACK. Quando si verifica tale evento, TCP confronta il valore di ACK con la propria variabile `SendBase`. La variabile di stato TCP `SendBase` è il numero di sequenza del più vecchio byte che non ha ancora ricevuto un acknowledgement. Di conseguenza SendBase -1 è il numero di sequenza dell'ultimo byte che si sa essere ricevuto correttamente e nell'ordin giusto. Tutto ciò è rappresentato nel seguente algoritmo:

Algoritmo 1 — *Algoritmo del mittente TCP*

```
1:  NextSeqNum ← InitialSeqNumber
2:  SendBase ← InitialSeqNumber
3:  loop (per sempre) do
4:      switch (evento) do
5:          evento: dati ricevuti dall'applicazione a livello superiore crea il segmento TCP con
            numero di sequenza NextSeqNum
6:              if il timer attualmente non è in funzione then
7:                  avvia il timer
8:              passa il segmento a IP
9:              NextSeqNum ← NextSeqNum + lunghezza(dati)
10:         evento: timeout del timer
11:             ritrasmetti il segmento che non ha ricevuto ACK con il più piccolo numero di
                sequenza
12:             avvia il timer
13:         evento: ACK ricevuto, con valore del campo ACK pari a y
14:             if y > SendBase then
15:                 SendBase ← y
16:                 if esistono attualmente segmenti senza ACK then
17:                     avvia il timer
```

L'ACK si riferisce a uno o più segmenti che precedentemente non avevano ricevuto conferma. Quindi, il mittente aggiorna la propria variabile `SendBase` e poi, se ci sono segmenti che necessitano di acknowledgement, riavvia il timer.

Scenari interessanti

Abbiamo visto coem TCP fornisca un trasferimento dati affidabile. Per comprendere come funziona il protocollo consideriamo qualche scenario. Consideriamo la seguente figura:

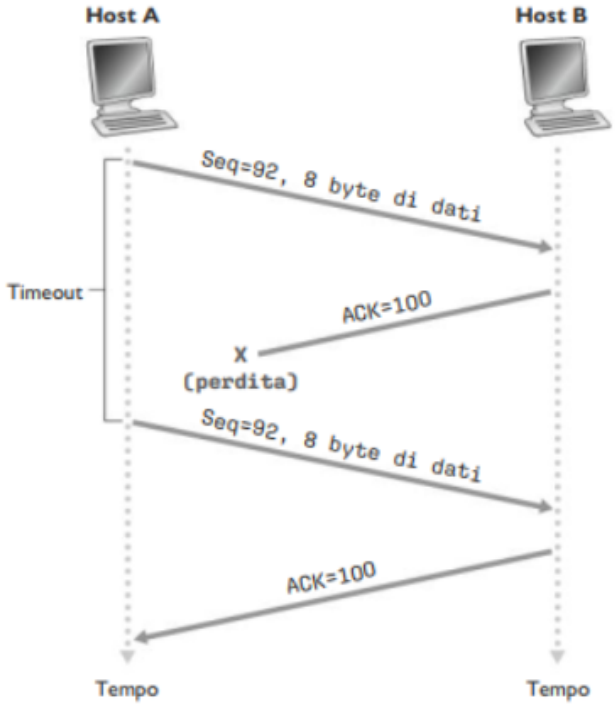


Figura 13: Ritrasmissione dovuta alla perdita di un acknowledgement

Questa figura mostra un caso in cui l'host A spedisce un segmento all'host B. Dopo aver inviato il segmento, A attende un segmento da B con numero di acknowledgement 100. Sebbene il segmento sia stato ricevuto da B, l'acknowledgement sul percorso inverso viene smarrito. In questo caso si verifica l'evento di timeout e l'host A ritrasmette lo stesso segmento. Quando l'host B riceve la ritrasmissione, rileva dal numero di sequenza che il segmento contiene dati che sono stati già ricevuti. Quindi, l'host B scarta i byte del segmento ritrasmesso e ritrasmette comunque l'ACK al mittente.

Consideriamo adesso la seguente figura.

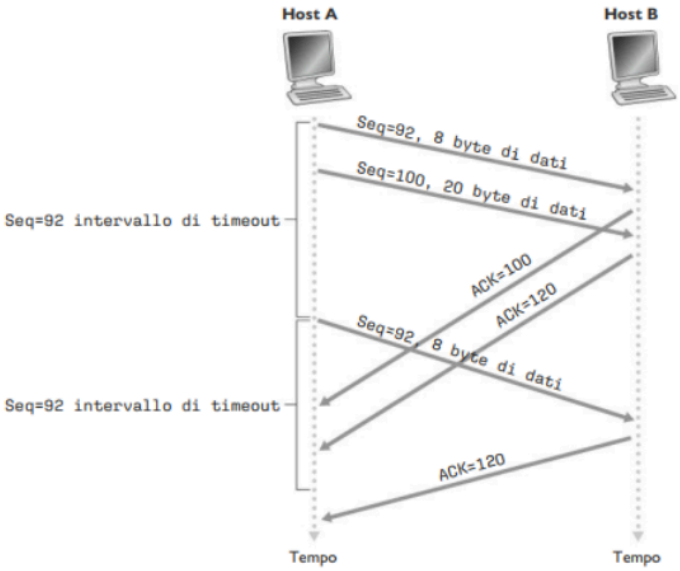


Figura 14: Segmento 100 non ritrasmesso

In questa figura A invia due segmenti. Supponiamo che entrambi arrivino intatti a B e che questo invii due acknowledgement separati per i segmenti, il primo numero 100, il secondo 120. Supponiamo che nessuno degli acknowledgement arrivi all'host A prima del timeout. Quando si verifica il timeout, l'host A rispedisce il primo segmento e riavvia il timer. Fino a quando l'ACK el secondo segmento non arriva prima el nuovo timeout, il secondo segmento non sarà ritrasmesso.
Consideriamo ora la seguente figura.

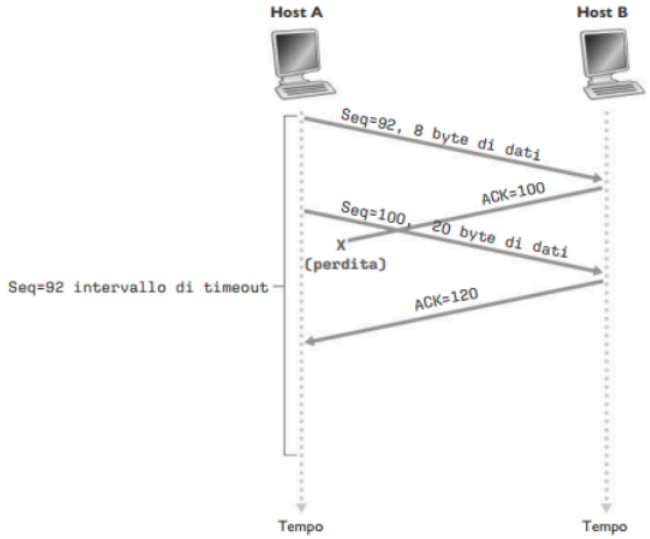


Figura 15: Acknowledgement cumulativo che evita le ritrasmessioni del primo segmento

In questa figura l'host A invia due segmenti, esattamente come nell'esempio precedente. L'acknowledgement del primo segmento viene perso nella rete ma, appena prima dell'evento di timeout, l'host A riceve un acknowledgement con numero 120. È pertanto, a conoscenza che l'host B ha ricevuto tutto fino al byte 119, quindi non rispedisce nessuno dei due segmenti.

Raddoppio dell'intervallo di timeout

Vediamo alcune varianti utilizzate dalla maggior parte delle implementazioni TCP. Las prima riguarda la lunghezza dell'intervallo di timeout dopo la scadenza di un timer. Con questa modifica, in tutti i casi in cui si verifica un timeout, TCP, ritrasmette il segmento con il più basso numeri di sequenza che non abbia ancora ricevuto acknowledgement. Ogni volta che questo si verifica, TCP imposta il successivo intervallo di timeout al doppio del valore precedente. Di conseguenza, gli intervalli crescono esponenzialmente a ogni ritrasmissione.
Questa modifica offre una forma limitata di controllo di congestione. La scadenza del timer viene causata, probabilemente, dalla congestione nella rete, ossia troppi pacchetti arrivano presso una (o più) code dei router nel percorso tra l'origine e la destinazione, provocando eliminazioni di pacchetti o lunghi ritardi di accodamento.

Ritrasmissione rapida

Uno dei problemi legati alle ritrasmissioni è che il periodo di timeout può rilevarsi relativamente lungo. Quando si smarrisce un segmento, il lungo periodo di timeout impone al mittente di ritardare il nuovo invio del pacchetto perso, aumentando il ritardo end-to-end. Però, il mittente potrebbe rilevare la perdita dei pacchetti ben prima che si verifichi l'evento di timeout grazie agli **ACK duplicati** relativi a un segmento il cui ACK è già stato ricevuto dal mittente.
Quando il destinatario TCP riceve un segmento con numero di sequenza superiore al successivo numero di sequenza atteso e in ordine, rileva un buco nel flusso di dati, ossia un segmento mancante. Tale vuoto potrebbe essere il risultato di segmenti persi o riordinati all'interno della rete. Il destinatario non può inviare un acknowledgement negativo esplicito al mittente, dato che TCP non lo prevede, ma si limita a mandare nuovamente un acknowledgement relativo all'ultimo byte di dati che ha ricevuto in ordine.
Dato che in molti casi il mittente invia un gran numero di segmenti, se uno di questi viene smarrito ci saranno molti ACK duplicati. Se il mittente TCP riceve tre ACK duplicati per lo stesso dato, consdiera questo evento come indice che il successivo segmento sia andato perduto. Nel caso incui siano stati ricevuti tre ACK duplicati, il mittente TCP effettua una **ritrasmissione rapida**, ripredendno il segmento mancante prima che scada il timer. In TCP con ritrasmissione rapida il

seguente frammento di codice sostituisce l'evento di ACK ricevuto nell'Algoritmo del mittente TCP.

ALGORITMO 2 — *Gestione ACK e Ristrasmissione Rapida*

```
1:  evento: ACK ricevuto, con valore del campo ACK pari a y
2:      if y > SendBase then
3:          SendBase ← y
4:          if esistono attualmente segmenti senza ACK then
5:              avvia il timer
6:      else
7:          incrementa il numero di ACK duplicati ricevuti per y
8:          if numero di ACK duplicati ricevuti per y = 3 then
9:
10:             rispedisci il segmento con numero di sequenza y
```

Go-Back-N (GBN) o ripetizione selettiva (SR)?

Ci chiediamo se TCP sia un protocollo GBN o SR. Ricordiamo che gli acknowledgement TCP sono cumulativi e che i segiemtni ricevuti correttamente, ma in modo disordinato, non vengono notificati singolarmente dal destinario. Quindi, il mittente TCP deve solo memorizzare il numero di sequenza più basso tra i byte trasmessi che non hanno ancora ricevuto acknowledgement (`SendBase`) e il numero di sequenza del successivo byte da inviare (`NextSeqNum`). In questo senso, TCP assomiglia molto a un protocollo di tipo GBN ma esistono molte differenze chiave. Molte implementazioni TCP memorizzano in un buffer i segmenti ricevuti correttamente, ma non in ordine. Consideriamo che il mittente invia una sequenza di segmenti $1, 2, ..., N$ che arrivano al destinatario in ordine e senza errori. Ipotizziamo che l'acknowledgement per il pacchetto $n < N$ vada perduto, ma che i restanti $N - 1$ giugano al mittente prima dei rispettivi timeout. In questo esempio GBN ritrasmetterebbe non solo il pacchetto n, ma anche tutti i pacchetti da $n + 1$ a N. TCP, invece, ritrasmetterebbe solo il segmento n. Inoltre, TCP nbon ritrasmetterebbe neppure questo segmento se ricevesse l'acknowledgement del segmento $n + 1$ prima della scandenza del timeout del segmento n.

Una modifica proposta di TCP, è l'**acknowledgement selettivo** che consente al destinatario di mandare acknowledgement in modo selettivo per i segmenti non in ordine anzichè cumulativamente per l'ultimo segmento ricevuto senza errori e nell'ordine giusto. Se combinato con la ritrasmissione selettiva, TCP è molto simile a un generico protocollo SR. Quindi, il meccanismo di ripristino degli errori è classifcato come un ibrido tra GBN e SR.

4.4.5. Controllo di flusso

Gli host agli estremi delle connessioni TCP riservano dei buffer di ricezione per la connessione. Quando la connessione TCP riceve byte corretti e in sequenza, li posiziona nel buffer di ricezione. Il processo applicativo associato legge i dati da questo buffer, ma non per forza nello stesso momento in cui vengono caricati. Se l'applicazione è relativamente letna nella lettura dei dati può accadere che il mittente mandi in overflow il buffer di ricezione inviando molti dati troppo rapidamente.
TCP offre un **servizio di controllo di flusso** alle proprie applicazioni per evitare che il mittente saturi il buffer del ricevente. Il controllo di flusso è pertanto un servizio di confronto sulla velocità, dato che paragona la frequenza di invio del mittente con quella di lettura dell'applicazione ricevente. I mittenti TCP possono anche essere rallentati dalla congestione nella rete IP. Questa forma di controllo del mittente viene detta **controllo di congestione**. Il controllo di flusso e controllo di congestione vengono gestiti in maniera simili ma sono causati da ragioni differenti.
TCP offre il controllo di flusso facendo mantenere al mittente una variabile chiamata **finestra di ricezione**, detta anche ***advertised window***,che fornisce al mittente un'indicazione dello spazio libero disponibile nel buffer del destinatario. Dato che TCP è full-duplex, i due mittenti mantengono finestre di ricezione distinte.
Supponiamo che l'host A stia inviando un file di grandi dimensioni all'host B su una connessione TCP. Quest'ultimo alloca un buffer di ricezione per la conessione, la cui dimensione è denotata con `RcvBuffer`. Definiamo le seguenti variabili.

- `LastByteRead`: numero dell'ultimo byte nel flusso di dati che il processo applicativo in B ha letto dal buffer;
- `LastByteRcvd`: numero dell'ultimo byte, nel flusso di dati, che proviene dalla rete e che è stato copiato nel buffer di ricezione di B.

Per non andare in overflow, dovremo per forza avere:

$$\text{LastByteRcvd} - \text{LastByteRead} \leq \text{RcvBuffer}$$

La finestra di ricezione, indicata con `rwnd`, viene impostata alla quantità di spazio disponibile nel buffer:

$$\text{rwnd} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$$

Adesso vediamo come viene utilizzata questa variabile. L'host B comunica all'host A quanto spazio disponibile sia presente nel buffer della connessione, scrivendo il valore corrente di `rwnd` nel campo apposito dei segmenti che manda ad A. L'host B inizializza `rwnd` con il valore di `RcvBuffer` e, ovviamente, deve tenere traccia di variabili specifiche per ogni connessione.
A sua volta, l'host A tiene traccia di due variabili, `LastByteSent` e `LastByteAcked`, il cui significato è rispettivamente «ultimo byte mandato» e «ultimo byte per cui si è ricevuto acknowledgement». La differenza tra queste esprime la quantità di dati

spediti da A per cui non si è ancora ricevuto un acknowledgement. Mantenendo questa quantità sotto il valore di `rwnd`, si garantisce che l'host A non mandi in overflow il buffer di ricezione dell'host B. Quindi l'host A si assicura che per tutta la durata della connessione sia rispettata la seguetne disuguaglianza:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

$$\text{EffectiveSendWindow} = \text{rwnd} - (\text{LastByteSent - LastByteAcked})$$

In questo schema esiste un problema tecnico secondario. Supponiamo che il buffer di ricezione dell'host B è pieno (`rwnd` $= 0$) e che, dopo averlo notificato all'host A, non abbia più nulla da inviare ad A. Quando il processo applicativo in B svuota il buffer, TCP non invia nuovi segmenti con nuovi valori di `rwnd`. Di conseguenza, l'host A non viene informato del fatto che si sia liberato un po' di spazio nel buffer di ricezione dell'host B. Per risolvere questo problema, le specifiche TCP richiedono che l'host A continui a inviare segmenti con un byte di dati quando la finestra di ricezione di B è zero. Il destinatario risponderà a questi segmenti con un acknowledgement. Prima o poi il buffer inizierà a svuotarsi e i riscontri conterranno un valore non nullo per `rwnd`.

4.4.6. Gestione della connessione TCP

Analizziamo come viene stabilita e rilasciata una connessione TCP. Supponiamo che un processo in esecuzione su un host, il client, voglia inizializzare una connessione con un altro processo in un altro host, il server. Il processo applicativo client informa il lato client di TCP di voler stabilire una connessione verso un processo nel server. Il TCP nel client procede a stabilire una connessione TCP con il TCP nel server nel modo descritto di seguito.

1. TCP lato client invia uno speciale segmento al TCP lato server. Questo segmento non contiene dati, ma uno dei bit nell'intestazione del segmento (Figura 11), il bit SYN, è posto a 1. Tale segmento viene detto segmento SYN. Inoltre il client sceglie a caso un numero di sequenza iniziale (`client_isn`) e lo pone nel campo numero di sequenza del segmento SYN inziale. Questo viene incapsulato in un datagramma IP e inviato al server.
2. Quando il datagramma IP contenente il segmento TCP SYN arriva all'host server, il server estrae il segmento dal datagramma, alloca il buffer e le variabili TCP alla connessione e invia un segmento di connessione approvata al client TCP. Questo segmento non contiene dati, ma nella sua intestazione ci sono tre informazioni importanti. Il bit SYN è posto a 1. Il campo ACK viene posto a `client_isn + 1`. Infine, il server sceglie il proprio numero di sequenza iniziale (`server_isn`) e lo pone nel campo numero di seguenza. Il segmento di connessione approvata viene detto **segmento SYNACK**
3. Alla ricezione del segmento SYNACK, anche il client alloca buffer e variabili alla connessione. L'host invia al server un altro segmento in risposta al segmento di

conenssione approvata dal server. Tale operazione viene svolta dal client ponendo il valore `server_isn + 1` nel campo ACK dell'intestazione del segmento TCP. Il bit SYN è posto a 0. In questo terzo passo dell'handshake a tre vie il campo dati del segmento può contenere informazioni che vanno dal client al server.

Questa procedura, come abbiamo detto, viene chiamata **handshake a tre vie**.

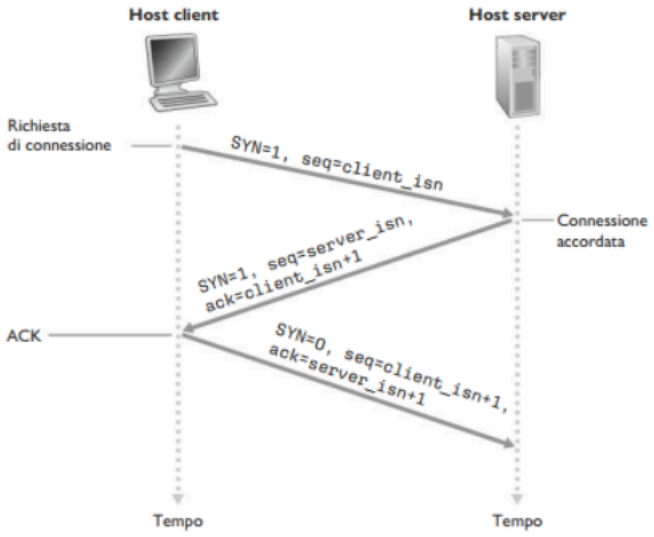


Figura 16: Handshake a tre vie di TCP

Ciasuno dei due processi che partecipano alla connessione può terminarla. In tal caso le risorse negli host vengono deallocate. Supponiamo che il client decida di chiudere la connessione. Il processo applicativo client invia un comando di chiusura, che forza il client TCP a inviare un segmento TCP speciale al processo server. Nell'intestazione di tale segmento troviamo il bit FIN con valore 1. Quando il server riceve questo segmento, rispondo inviando un acknowledgement al client. Il server spedisce il proprio segmento di shutdown, con il bit FIN pari a 1. Infine, il client manda a sua volta un acknowledgement a quest'ultimo segmento del server. A questo punto, tutte le risorse degli host risultano deallocate.
Nell'aco di una connessione TCP, i protocolli TCP attraverso vari **stati TCP**.
Gli stati attraversati da un client TCP sono raffigurati nella seguente figura:

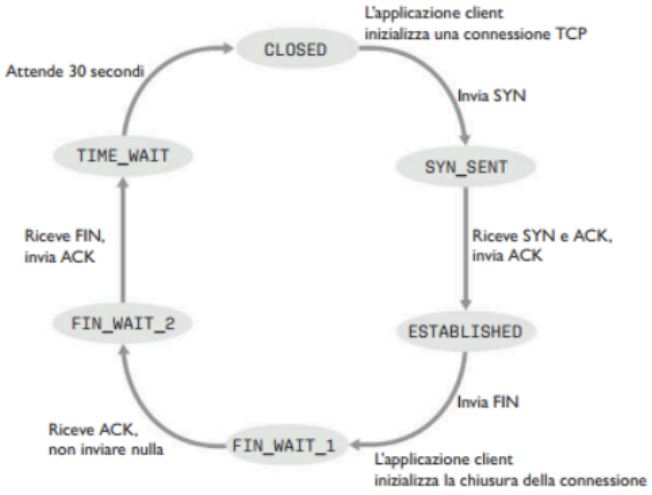


Figura 17: Sequenza di stati visitati da un client TCP

Gli stati attraversati da un server TCP sono rappresentati nella seguente figura:

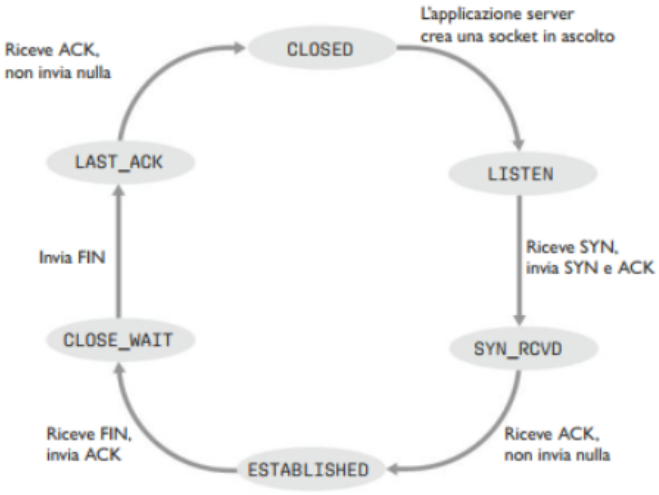


Figura 18: Sequenza di stati visitati da un server TCP

Esiste anche un'altra flag per chiudere una connessione TCP e si tratta del flag **RST**. Il flag RST (reset) interrompre bruscamente una connessione, senza deallocare buffer e variabili. Ad esempio, se arriva un SYN verso una porta su cui nessun processo è in ascolto, l'host risponde con un segmento con RST = 1 che sta ad indicare che non c'è una socket per quel segmento e quindi di non ritrasmettere il messaggio.

4.4.7. Controllo di congestione TCP

La ritrasmissione dei pacchetti, in caso di perdita di questi, tratta un sintomi della congestione di rete, ma non le cause della congestione di rete: il tentativo da parte di troppe sorgenti di invidare dati a ritmi troppo elevati. In questo caso sono richiesti meccanismi per adeguare l'attività dei mittenti in relazione al traffico.
Il protocollo TCP utilizza il controllo di congestione end-to-end anzichè il controllo di congestione assistito dalla rete, in quanto il livello IP non fornisce ai sistemi perficerici alcun feedback per quanto riguarda la congestione della rete.

Controllo di congestione TCP classico

L'approccio scelto da TCP consiste nell'imporre a ciascun mittente un limite al tasso di invio sulla propria connessione in funzione della congestione di rete percepita. Se il mittente TCP si accorge di condizioni di scarso traffico sul percorso che porta alla destinazione, incrementa il proprio tasso di trasmissione, altrimenti, lo riduce.
Dobbiamo capire:

1. come il mittente TCP limita il tasso di invio del traffico sulla propria rete;
2. come percepisce congestione sul percorso che porta alla destinazione;
3. quale algoritmo dovrebbe essere usto dal mittente per variare il tasso di invio in funzione della congestione end-to-end.

Capiamo prima come TCP possa ridurre la velocità di invio del traffico sulla proprio connessione. Sappiamo che gli estremi di una connessione TCP gestiscono un buffer di ricezione, uno di invio e diverse variabili tra cui `LastByteRead` e `rwnd`.
Il meccanismo di controllo di congestione TCP fa uso di una nuova variabile: la **finestra di congestione**, indicata con `cwnd`, che impone un vincolo alla velocità di immissione di traffico sulla rete da parte del mittente. Nello specifico, la quantità di dati che non hanno ancora ricevuto acknowledgement da un mittente non può eccedere il minimo tra i valori di `cwnd` e `rwnd`, ossia:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min(\text{cwnd}, \text{rwnd})$$

$$\text{EffectiveSendWindow} = \min(\text{cwnd}, \text{rwnd}) - (\text{LastByteSent - LastByteAcked})$$

Assumiamo che il buffer di ricezione sia sufficientemente capiente da poter ignorare il limite sulla finestra di ricezione; pertanto, la quantità di dati ceh non hanno ricevuto acknowledgement è limitata soltanto da `cwnd`. Assumiamo, inoltre, che il mittente abbia sempre dati da inviare, per cui la finestra di congestione è sempre in uso.
Il vincolo sopra citato limita il tasso di trasmissione del mittente soltanto in modo indiretto. Consderiamo una connessione in cui la perdita di pacchetti e i ritardi di trasmissione siano trascurabili. Quindi, all'inizio di ogni RTT, il vincolo consente al mittente di trasmettere `cwnd` byte di dati sulla connessione; al termine del RTT il mittente riceve gli acknowledgement relativi ai dati. Il tasso di invio del mittente è

approssivamente $\frac{\text{cwnd}}{\text{RTT}}$ byte/s. Modificando `cwnd` il mittente può regolare la velocità di invio dei dati sulla propria connessione.
Consideriamo ora come il mittente TCP percepisce la presenza di congestione sul percorso verso la destinazione. Definiamo «evento di perdita» per il mittente TCP l'occorrenza o di un timeout o della ricezione di tre ACK duplicati da parte del destinatario. In presenza di una congestione eccessiva, uno o più buffer dei router lungo il percorso vanno in overflow, causando l'eliminazione di un datagramma. Il datagramma eliminato, costituisce un evneto di perdita presso il mittente che lo considera come un'indicazione di congestione sul percorso tra se e il destinatario.
Quando gli acknowledgement relativi ai vari segmenti verranno ricevuti dal mittente TCP, TCP interpreta l'arrivo di essi come un'indicazione che tutto va bene e utilizza gli acknowledgement per aumentare l'ampiezza della propria finestra di congestione (e quindi, la velocità di trasmissione). Se gli acknowledgement arrivano con frequenza relativamente bassa, allora la finestra di congestione verrà ampliata piuttosto lentamente. Al contrario, la finestra di congestione verrà ampliata più rapidamente. Per questo si dice che TCP è **auto-temporizzato** (*self-clocking*).
Dobbiamo capire come un mittente TCP debba determinare il tasso a cui dovrebbe trasmettere. Se tutti i mittenti TCP trasmettessero troppo velocemente potrebbero congestionare la rete. Se, invece, i mittenti TCP trasmettessero troppo lentamente potrebbero sottoutilizzare l'ampiezza di banda della rete; potrebbero trasmettere a tasso più elevato senza congestionare la rete.
Possiamo analizare i dettagli dell'**algoritmo di controllo di congestione TCP** che presenta tre fasi principali:
1. *slow start*;
2. *congestion avoidance*;
3. *fast_recovery.*

Slow start e congestion avoidance sono componenti obbligatorie di TCP e differiscono nel modo in cui aumentano la grandenza di `cwnd` in risposta agli acknowledgement ricevuti. Slow start aumenta la dimensione di `cwnd` molto più rapidamente di congestion avoidance. La fast recovery è suggerita, ma non obbligatoria, per i mittenti TCP.

Slow start
Quando si stabilisce una connessione TCP, il valore di `cwnd` viene in genere inizializzato a 1 MSS, l che comporta una velocità di invio iniziale di circa $\frac{\text{MSS}}{\text{RTT}}$.
Dato che la banda disponibile può essere molto più grande di $\frac{\text{MSS}}{\text{RTT}}$, il mittente TCP gradirebbe scoprire velocemente la banda disponibile. Quindi, durante la fase iniziale, detta **slow start**, il valore di `cwnd` parte da 1 MSS e si incrementa di 1 MSS ogni volta che un segmento trasmesso riceve un acknowledgement. Questo processo ha come effetto il raddoppio della velocità di trasmissione a ogni RTT.

Quindi la velocità di trasmissione parte lentamente, ma cresce in modo esponenziale durante la fase di slow start.
Se c'è un evento di perdita indicato da un evento di timeout, il mittente TCP pone il valore di `cwnd` pari a 1 e inizia di nuovo il processo di slow start. Inoltre, pone il valore di una seconda variabile di stato, `ssthresh` (*soglia di slow start*) a $\frac{\text{cwnd}}{2}$.
Il secondo modo in cui la fase di slow start può terminare è legato direttamente al valore di `ssthresh` . Poichè `ssthresh` è impostato a metà del valore di `cwnd` all'ultimo rilievo di congestione, potrebbe essere temerario continuare a raddoppiare il valore di `cwnd` quando raggiunge o sorpassa il valore di `ssthresh` . Quindi, quando cwnd = ssthresh, la fase di slow start termina e TCP entra in modalità di congestion avoidance.
Inizialmente il valore di `ssthresh` viene inizializzato a un valore costante arbitrariamente alto e solo dopo il primo evento di perdita viene impostato a $\frac{\text{cwnd}}{2}$. L'ultimo modo in cui la fase di slow start può terminare è quando vengono rilevati tre acknowledgement duplicati, nel qual caso TCP opera una ritrasmissione rapida ed entra nello stato di fast recovery.
Il comportamento di TCP in questa fase è racchiuso in questo automa:

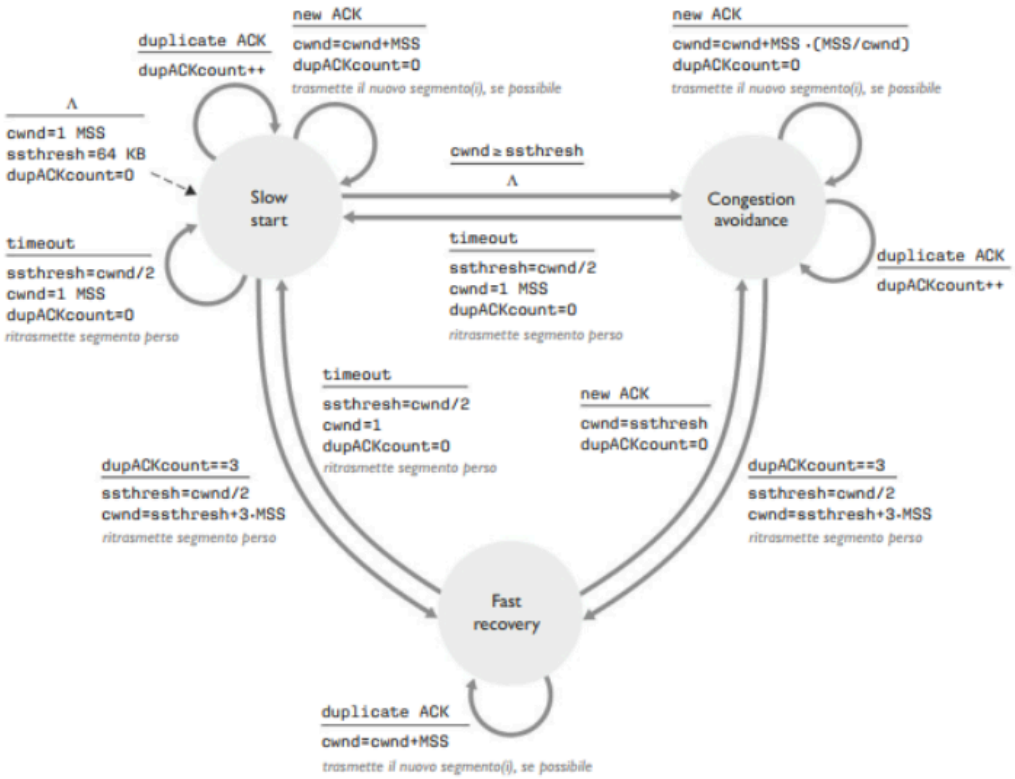


Figura 19: Descrizione tramite DFA del controllo di conestione TCP

Congestion avoidance

Quando TCP entra nello stato di congestion avoidance, il valore di `cwnd` è circa la metà di quello che aveva l'ultima volta in cui era stata rilevata la congestione. Quindi, invece di raddoppiare il valore di `cwnd` ogni RTT, TCP adotta un approccio più conservativo, incrementando `cwnd` di 1 MSS ogni RTT. Ciò si può ottenere in diversi modi: un approccio comune è l'incremento da parte del mittente TCP della propria `cwnd` di MSS $\times \left(\frac{\text{MSS}}{\text{cwnd}}\right)$ byte ogni qualvolta riceva un nuovo acknowledgement.

Dobbiamo capire quando termina l'incremento lineare (1 MSS per RTT) durante la congestion avoidance. L'algoritmo di congestion avoidance, quando si verifica un timeout, si comporta nello stesso modo di slow start: il valore di `cwnd` è posto uguale a 1 MSS e il valore di `ssthresh` viene impostato alla metà del valore di `cwnd` al momento del timeout. In caso di acknowledgement duplicati TCP dimezza il valore di `cwnd` (aggiungendo 3 MSS per tenere conto dei duplicati) e imposta il valore di `ssthresh` pari a metà del valore di `cwnd` al momento del ricevimento dei tre ACK duplicati. Infine, TCP entra nello stato di fast recovery.

Fast recovery

Durante la fase di fast recovery il valore di `cwnd` è incrementato di 1 MSS per ogni ACK duplicato ricevuto relativamente al segmento perso che ha causato l'entrata di TCP nello stato di fast recovery. Infine, quando arriva un ACK per il segmento perso, TCP entra nello stato di congestion avoidance dopo aver ridotto il valore di `cwnd`. Se si verifica un timeout vi è invece una transizione dallo stato di fast recovery a quello di slow start dopo aver effettuato le stesse azioni presenti sia in slow start che in congestion avoidance

$$\text{ssthresh} = \frac{\text{cwnd}}{2}$$
$$\text{cwnd} = 1 \text{ MSS}$$

Fast recovery è un componente raccomandato ma non obbligatorio. Una prima versione di TCP, nota come **TCP Tahoe**, portava in modo incondizionato la finestra di congestione a 1 MSS ed entrava nella fase di slow start dopo qualsiasi evento di perdita. La versione più recente, **TCP Reno**, adotta invece fast recovery. Una versione alternativa di TCP Reno è **TCP NewReno** che introduce un'evoluzione intelligente dell'algoritmo di Fast Recovery per gestire le perdite multiple di pacchetti senza uscire prematuramente dalla fase di recupero.

- **Riconoscimento dell'ACK parziale**: NewReno sa distinguere chiaramente tra un ACK parziale (che conferma solo una parte di dati) e un ACK cumulativo definitivo (conferma tutti i dati).
- **Permanenza in Fast Recovery**: qundo riceve un ACK parziale, NewReno comprende che si è perso un altro pacchetto oltre al primo. Invece di uscire da

Fast Recovery, il mittente ritrasmette immediatamente il pacchetto successivo indicato dall'ACK parziale e rimane in Fast Recovery.

- **Uscita corretta**: la fase di Fast Recovery termina solo quando arriva un ACK comulativo che conferma la ricezione di tutti i dati che erano stati spediti prima di entrare in Fast Recovery.
- Tramite l'utilizzo della variabile `recover`, entrando in fast recovery NewReno salva in `recover` il numero dell'ultimo segmento inviato fino a quel momento e rimane in fast recovery finchè un ACK non riscontra tutti i dati fino a `recover`.

La Figura 19 mostra la descrizione completa degli algoritmi di congestione (slow start, congestion avoidance e fats recovery) tramite un automa a stati finiti. La figura indica anche in quali punti può avvenire la trasmissione di nuovi segmenti o la ritrasmissione di segmenti già inviati.

TCP CUBIC

Dato l'approccio al controllo di congestione di TCP Reno con aumento additivo e decremento moltiplicativo, ci si chiede se questo sia il metodo migliore per trovare una velocità di trasmissione dei pacchetti appena inferiore alla soglia di attivazione della perdita dei pacchetti. Infatti una strategia che dimezza la velocità di invio, e quindi la aumenta piuttosto lentamente, può essere eccessivamente cauta. Se lo stato del collegamento congestionato non è cambiato molto, forse è meglio aumentare la velocità di invio rapidamente per avvicinarsi a quella pre-perdita dei pacchetti e quindi sondare con cautela la larghezza di banda. Quest'approccio è al centro di una versione di TCP nota come **TCP CUBIC**.

In TCP CUBIC la finestra di congestione viene aumentata solo alla ricezione di un ACK e nelle flasi di slow start e fast recovery rimane la stessa. CUBIC modifica solo la fase di congestion avoidance nel modo descritto di seguito:

- Sia $W_{\max}$ la dimensione della finestra del controllo di congestione di TCP nell'istante in cui viene rilevata l'ultima perdita e sia K l'istante nel futuro in cui la dimensione della finestra di TCP CUBIC raggiungerà nuovamente $W_{\max}$. Diversi parametri di CUBIC determinano il valore di K che indica quanto velocemente la dimensione della finestra di congestione del protocollo raggiungerebbe $W_{\max}$.
- CUBIC aumenta la finestra di congestione in funzione del cubo della distanza tra l'istante corrente t e K. Pertanto quando t è distante da K, gli incrementi sono molto maggiori rispetto a quando t è vicino. Quindi CUBIC aumenta rapidamente la velocità per avvicinarsi alla velocità di pre-perdita $W_{\max}$ e solo quando si avvicina a $W_{\max}$ esamina con cautela la larghezza di banda. Una volta superato $W_{\max}$ la velocità torna a crescere rapidamente per cercare nuova banda disponibile.

- La regola cubica consente a CUBIC di trovare più rapidamente un nuovo punto di lavoro qualora il livello di congestione del collegamento che ha causato la perdita sia cambiato in modo significativo.

Dopo una perdita in CUBIC `cwnd` scende a $0.7 \cdot \text{cwnd}$ e non alla metà come in Reno. TCP CUBIC è attualmente ampiamente utilizzato.

4.4.8. Notifica esplicita di congestione dalla rete

L'*Explicit Congestion Notification (ECN)* è la forma di controllo di congestione assistita dalla rete effettuata in Internet. Sono coinvolti sia TCP sia IP. A livello di rete vengono utilizzati due bit nel campo *Tipo di Servizio* dell'intestazione IP. Se un router è congestionato, imposta tali bit e invia il pacchetto IP contrassegnato al destinatario, che quindi informa il mittente.
ECN permette di segnalare la congestione prima che si verifichino perdite di pacchetti.
I bit ECN vengono inoltre utilizzati dal mittente per segnalare che mittente e ricevente sono abilitati all'uso di ECN. Il mittenete imposta a 01 o 10 per indicare il supporto a ECN, mentre il router intermedio che rileva congestione li marca con 11. Quando il destinatario TCP riceve indicazione di congestione ECN, ne informa il mittente TCP impostando il bit ECE (*Explicit Conegestion Notification Echo*) all'interno di un segmento ACK. Il mittente TCP reagisce dimezzando la finestra di congestione e imposta il bit CWR (*Congestion Window Reduced*) nell'intestazione el successivo segmento che invia al ricevente.
Il destinatario TCP imposta il flag `ECE = 1` in tutti gli ACK successivi finchè non riceve dal mittente un segmento con il flag `CWR = 1`. Altri protocolli a livello di trasporto usano il protocollo ECN, *Datagram Congestion Control Protocol (DCCP)* che fornisce un controllo di congestione ECN leggero, ma non affidabile. Mentre il protocollo DCTCP (*Data Center TCP*) è progettato specificatamente per infrastrutture di rete all'interno di data center.
Anche DCQCN (Data Center Quantized Congestion Notification), progettato specificatamente per le reti dei data center, si avvale dell'ECN.

4.4.9. Fairness

Consideriamo K connessioni TCP, ciascuna con differente percorso end-to-end, ma che passano tutte da un collegamento con capacità trasmissiva di R bps; con ciò intendiamo che tutti gli altri collegamenti siano congestionati e dispongano di elevata capacità trasmissiva.
Si dice che un meccanismo di controllo di congestione è **fair** se la velocità di trasmissione media di ciascuna connessione è approssimativamente $\frac{R}{K}$; cioè che ciascuna connessione ottiene la stessa porzione di banda del collegamento.

Fairness e UDP

Il meccanismo della finestra di congestione consente al controllo di congestione TCP di regolare il tasso di trasmissione delle applicazioni. Per questo molte applicazioni multimediali non fanno uso di TCP: non vogliono che il loro tasso di trasmissione venga ridotto. Queste applicazioni preferiscono utilizzare UDP, che non incorpora il controllo di congestione, preferendo di immettere i propri dati a frequenza costante e occosionalmente perdere pacchetti piuttosto che non perderli, ma dover ridurre il loro tasso di trasmissione a livelli equi nei momenti ditraffico. Le applicazioni che fanno uso di UDP non sno fair: non cooperano con altre nè adeguano il loro tasso di trasmissione in modo appropriato. Per questo motivo le sorgenti UDP possono soffocare il traffico TCP e per questo sono stati proposti meccanismi di controllo di congestione capaci di evitare che il traffico UDP blocchi il throughput di Internet.

Fairness e connessioni TCP parallele

Anche se UDP si comportasse in modo equo, il problema della fairness non sarebbe completamente risolto, perchè un'applicazione basata su TCP potrebbe usare più connessioni in parallelo. Spesso i browser usano più connessioni TCP in parallelo per trasferire il contenuto elle pagine. Le applicazioni che usano connessioni in parallelo ottengono una porzione di banda maggiore sui collegamenti congestionati. Dato che il traffico web è molto diffuso, le connessioni in parallelo non sono rare.

4.5. QUIC: Quick UDP Internet Connection

Se i modelli di servizio offerti da UDP e TCP non sono adatti ai serviiz del livello di trasporto richiesti da un'applicazione, gli sviluppatori dell'applicazione possono sempre usare un proprio protocollo a livello di applicazione. Questo può succedere anche quando un'applicazione necessita dei servizi forniti da UDP, ma non desidera tutte le funzionalità richieste da TCP o richiede servizi diversi da quelli forniti da TCP. Questo è l'approccio adottato nel protocollo **QUIC**. QUIC è un nuovo protocollo a livello di applicazione, progettato da zero, per migliorare le prestazioni del livello di trasporto per HTTP sicuro.

QUIC utilizza molti degli approcci per il trasferimento dati affidabile, il controllo di congestione e la gestione della connessione affrontata per i protocolli a livello di trasporto.

QUIC utilizza UDP come protocollo del livello di trasporto ed è progettato per interfacciarsi ad una versione semplificata, ma evoluta, di HTTP/2. Adesso è stato standardizzato ufficialmente come HTTP/3. Alcune caratteristiche principali di QUIC sono le seguenti.

- **Orientato alla connessione e sicuro**. QUIC è un protocollo orientato alla connessione tra due endpoint. Ciò richiede una fase di handshaking tra gli endpoint. Due parti dello stato di connessione sono gli ID dell'origine e della

destinazione della connessione. Tutti i pacchetti QUIC sono crittografati e QUIC combina l'handshake e la cifratura. In questo modo richiede 1 RTT prima di inviare i dati, in TCP invece sono necessari 2 RTT. La cifratura del traffico è **obbligatoria** ed è basata su TLS.

- **Flussi (Streams)**. QUIC consente il multiplexing di diversi flussi a livello di applicazione attraverso una singola connessione QUIC; inoltre, una volta stabilita una connessione QUIC, nuovi flussi possono essere raggiunti rapidamente. Un flusso è un'astrazione per la consegna bidirezionale affidabile e in ordine dei dati tra due endpoint QUIC. In HTTP/3, per ogni oggetto in una pagina web ci sarebbe un flusso diverso. Ogni connessione ha un ID e ogni flusso all'interno di una connessione ha un ID di flusso; entrambi questi ID sono contenuti in un'intestazione del pacchetto QUIC.
- **Trasferimento dati affidabile e con controllo di congestione**. QUIC fornisce un trasferimento dati affidabile a ogni flusso QUIC separatamente. Dal momento che TCP fornisce un servizio di consegna dei byte affidabile e in ordine, le richieste HTTP multiple devono essere consegnate in ordine al server HTTP destinazione. Pertanto, se i byte di una richiesta HTTP vengono persi, le restanti richieste HTTP non possono essere consegnate finchè tali byte non vengono ritrasmessi e ricevuti correttamente da TCP sul server HTTP: il cosidetto problema del blocco HOL affrontato in precedenza. Poichè QUIC fornisce una consegna in ordine e affidabile per flusso, un segmento UDP perso influisce solo sui flussi i cui dati sono stati trasportati in tale segmento; i messaggi HTTP di altri flussi possono continuare a essere ricevuti e consegnati dall'applicazione. QUIC fornisce un trasferimento dati affidabile utilizzando meccanismi di acknowledgement simili a quelli di TCP. Il controllo di congestione di QUIC si basa su TCP NewReno, una piccola variante del protocollo TCP Reno.